Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 5

Disciplina: Tehnici de programare

Tema: ”Laborator Python: funcții, metode și importuri pe web”

A efectuat : st. gr.SI – 263; Topală Dorofei

A verificat : asistent universitar: Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

- Studiul și utilizarea modulelor standard din Python (requests, urllib, re, html.parser, hashlib, json, socket, ssl) pentru realizarea de cereri HTTP, analiza antetelor de securitate, parsarea paginilor HTML, interogări DNS, verificarea certificatelor SSL și crearea unei aplicații modulare împărțite în webtools.py și main.py pentru auditul unui site web.

1. Partea 1: Importuri

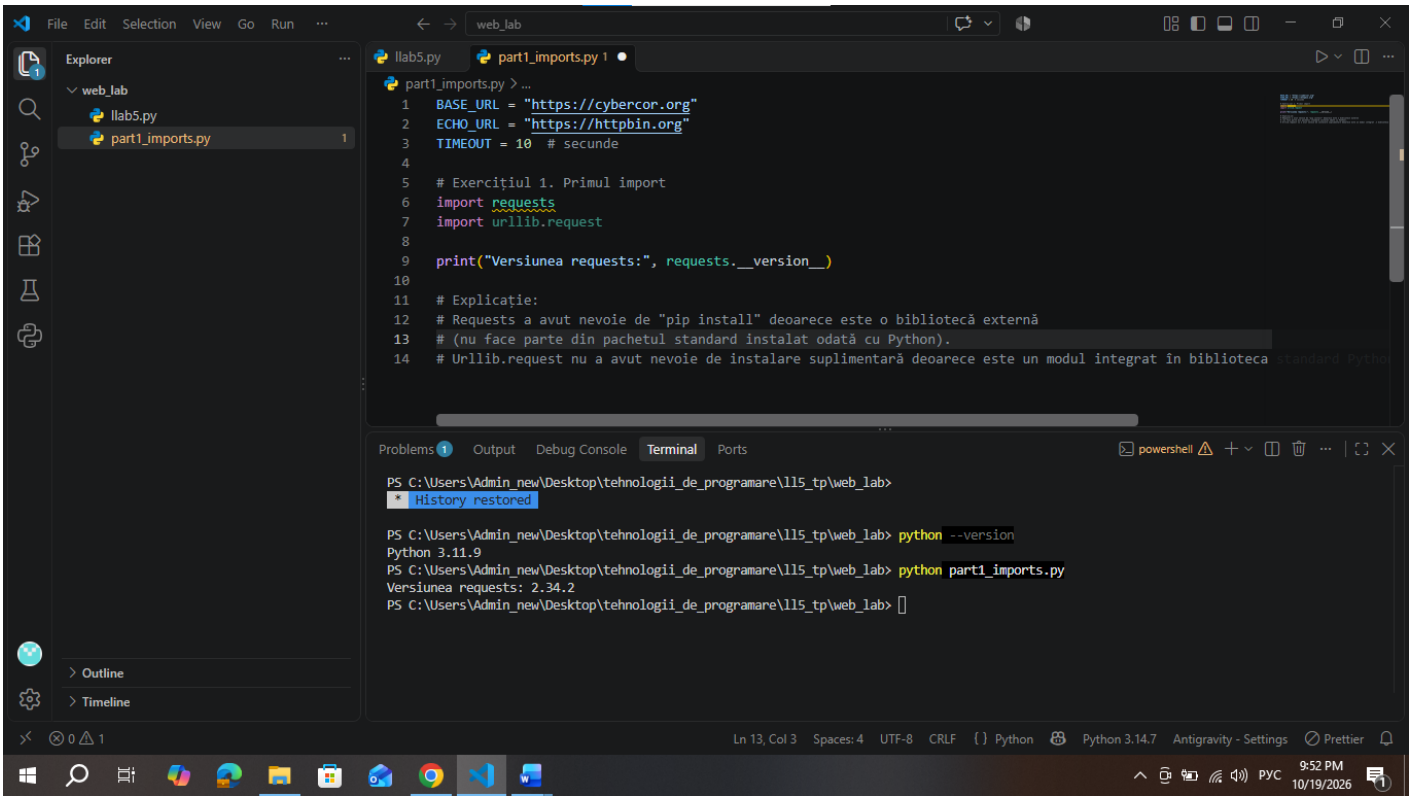


Figura 1.1 - Primul import

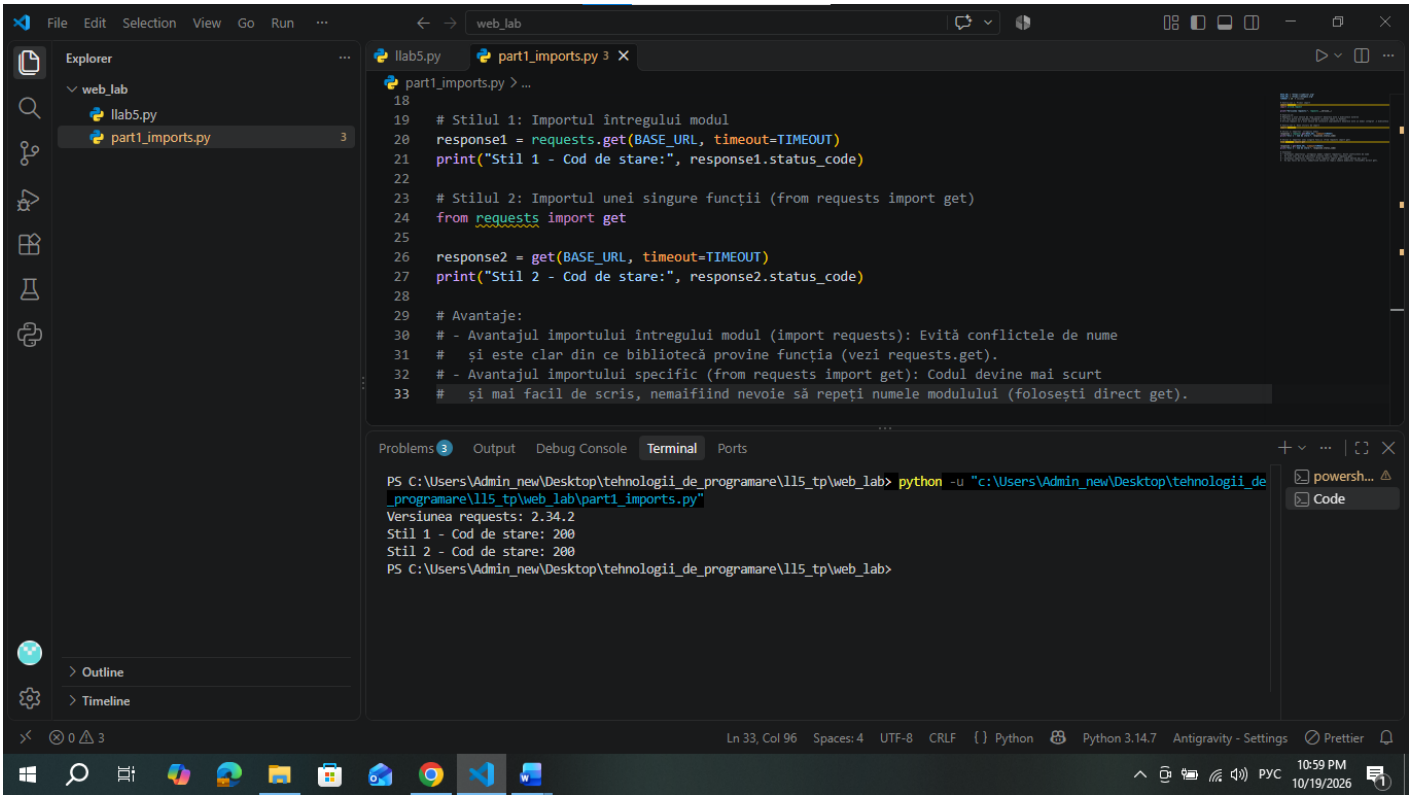


Figura 1.2 - Două stiluri de import

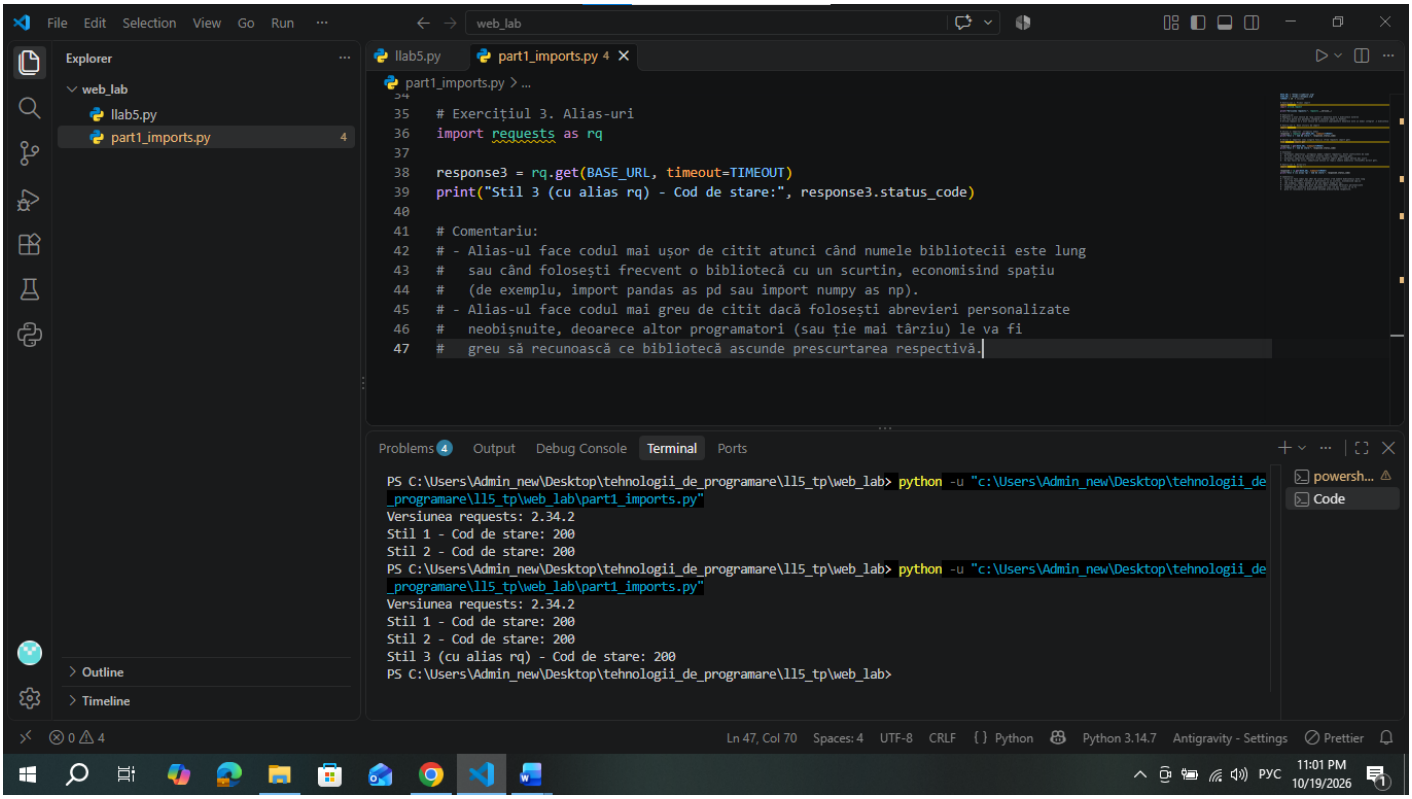


Figura 1.3 - Alias-uri

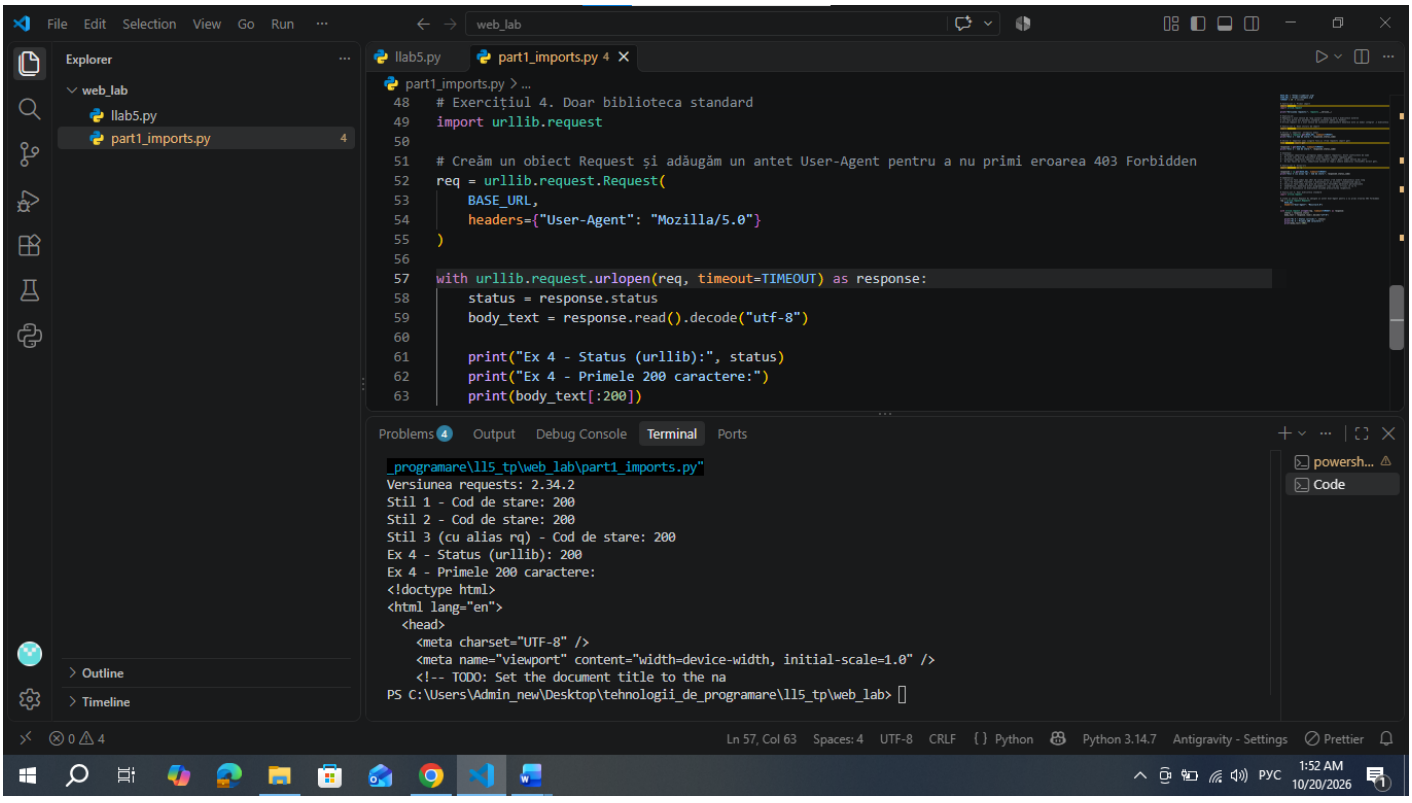


Figura 1.4 - Doar biblioteca standard

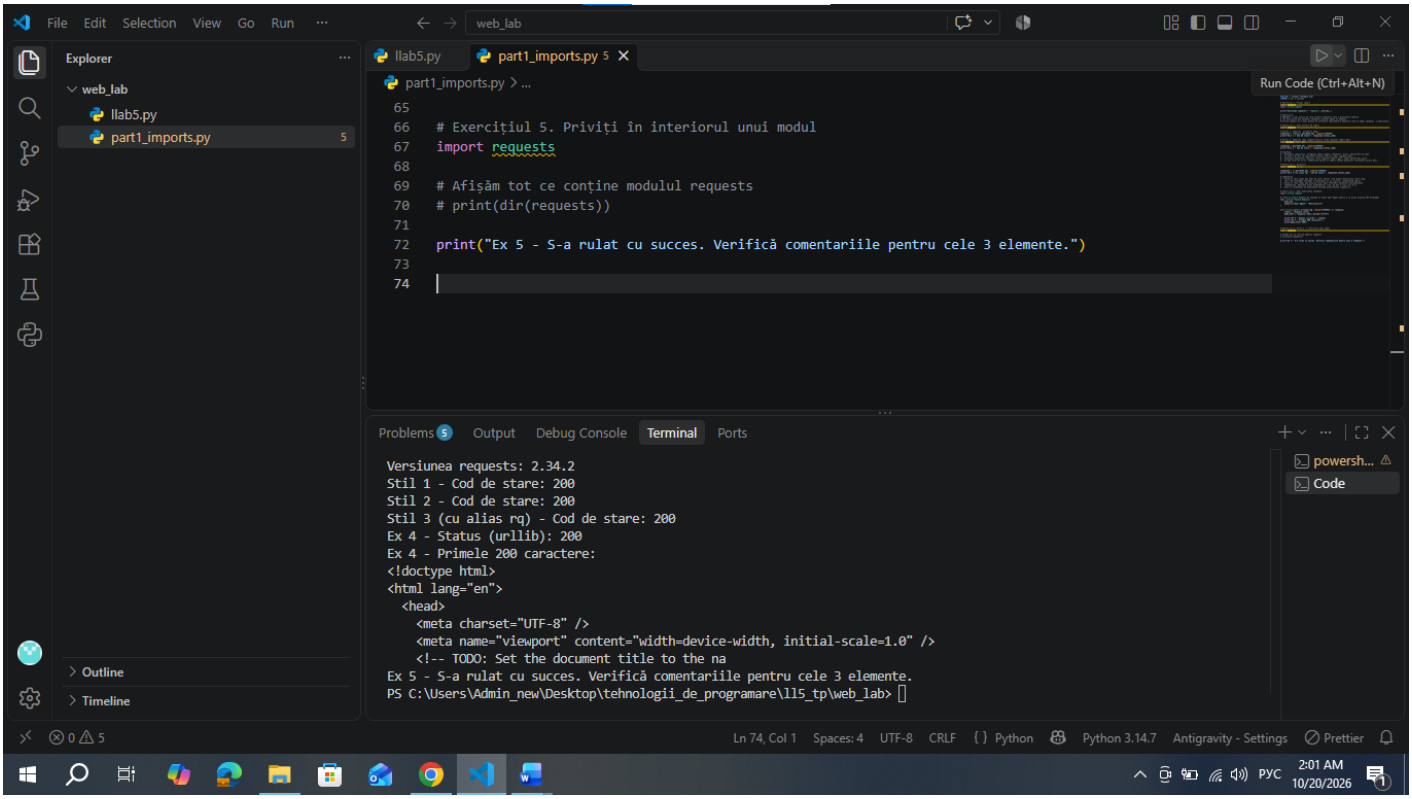


Figura 1.5 - Priviți în interiorul unui modul

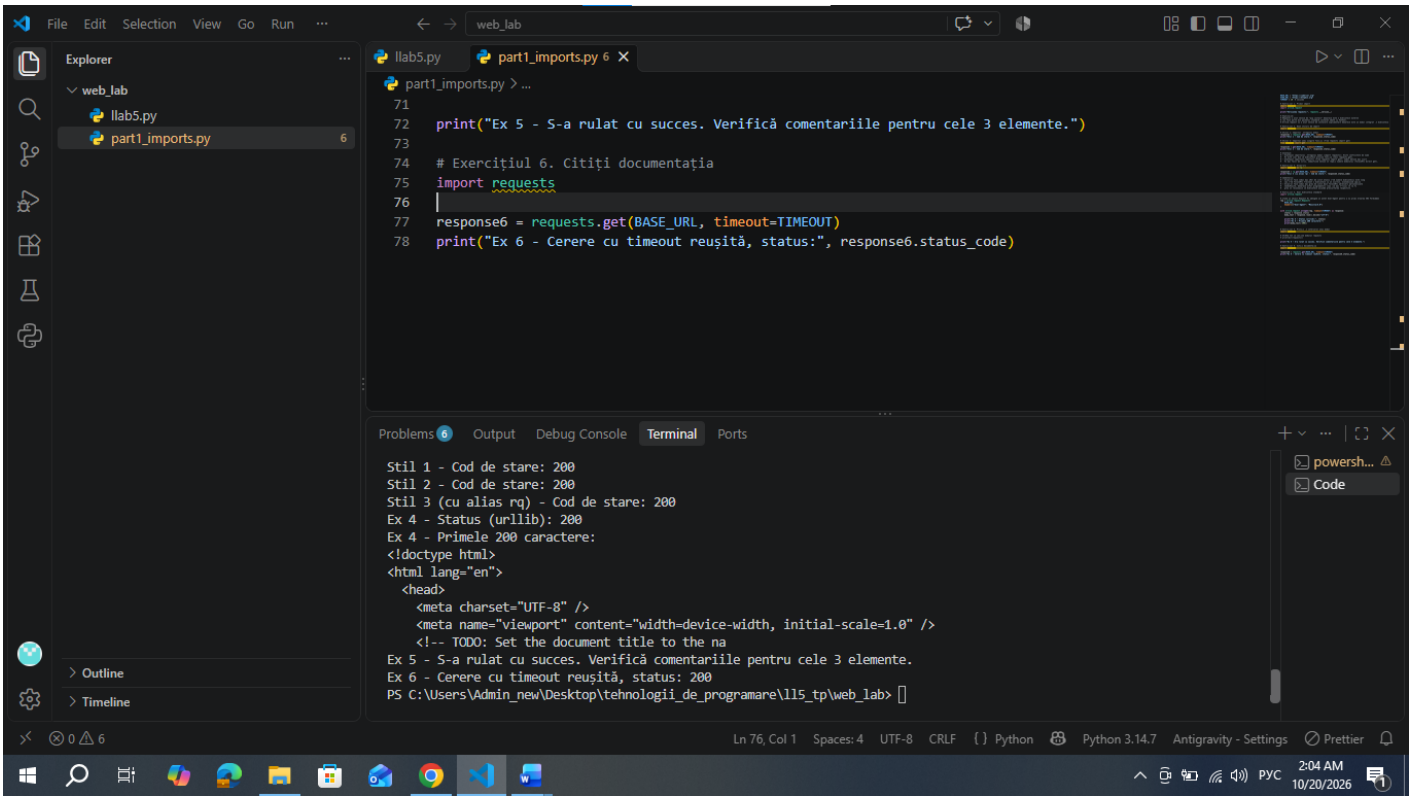


Figura 1.6 - Citiți documentația

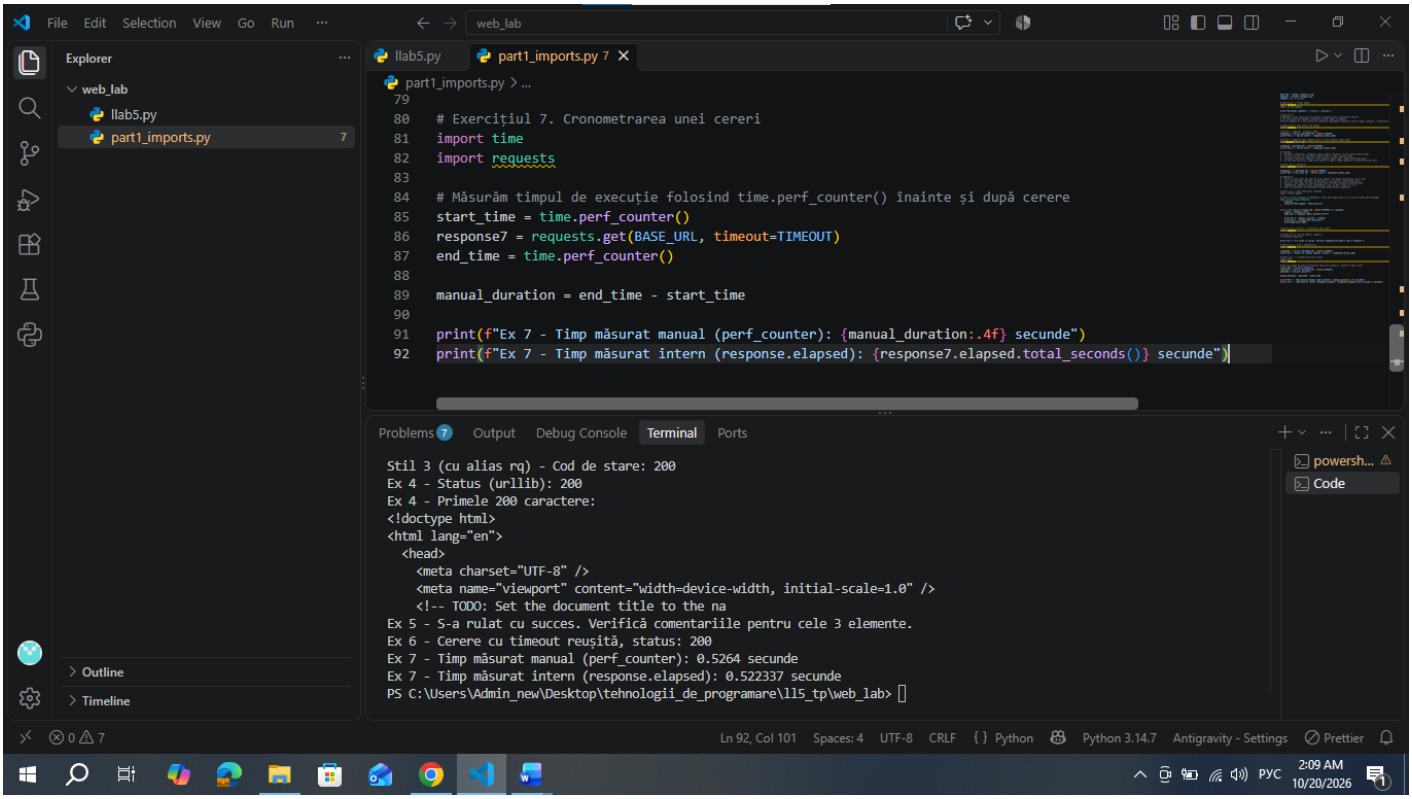


Figura 1.7 - Cronometrarea unei cereri

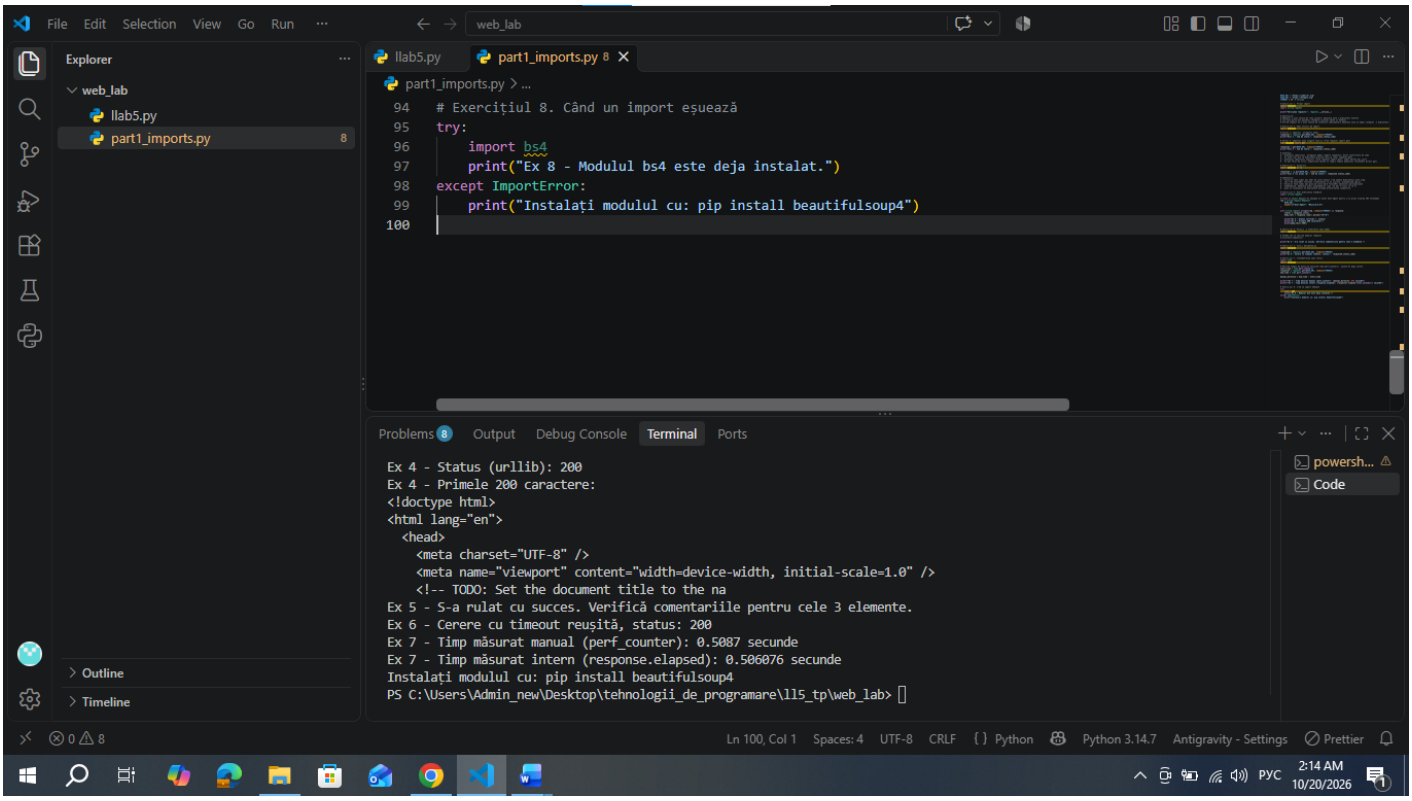


Figura 1.8 - Când un import eșuează

2. Partea 2: Metode și atribute ale obiectelor web:

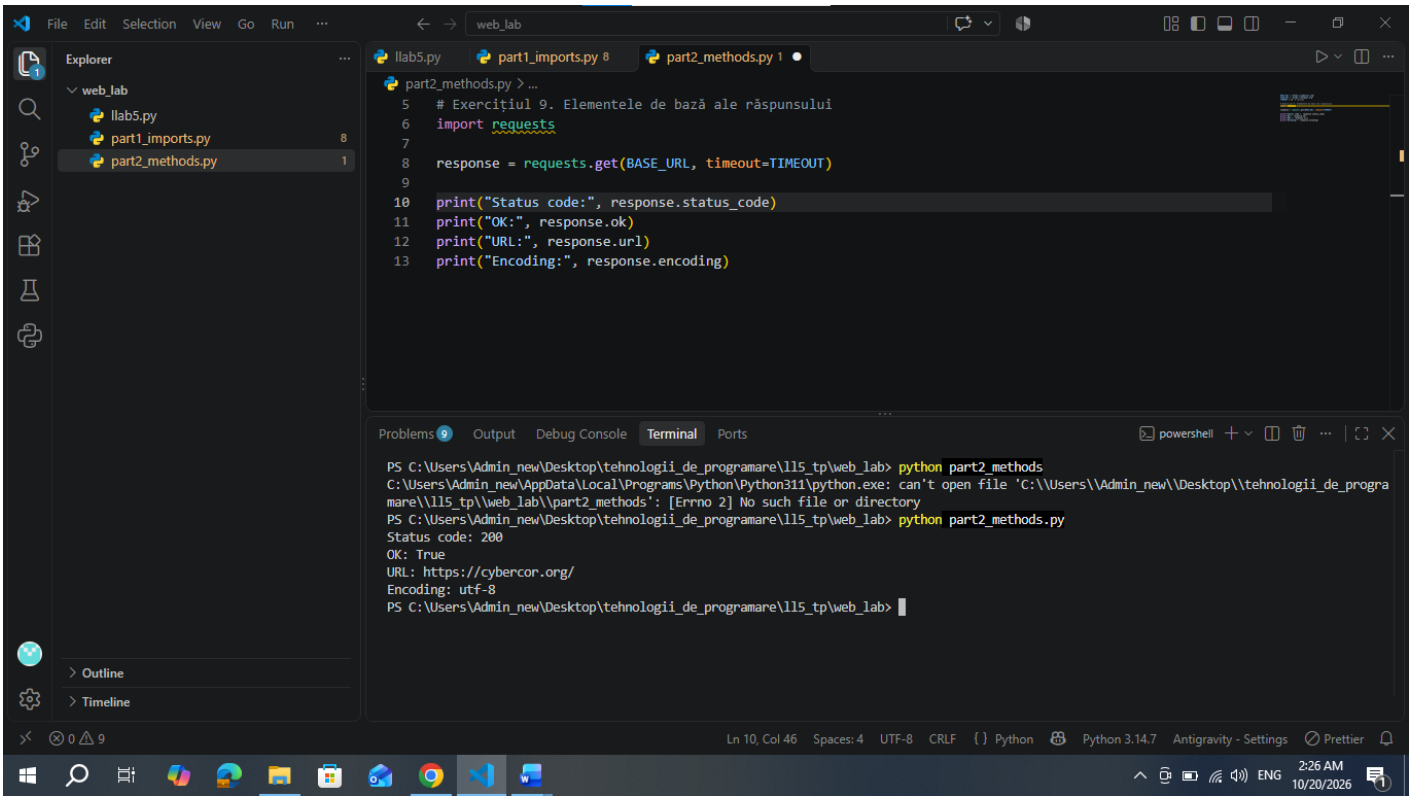


Figura 2.1 - Elementele de bază ale răspunsului

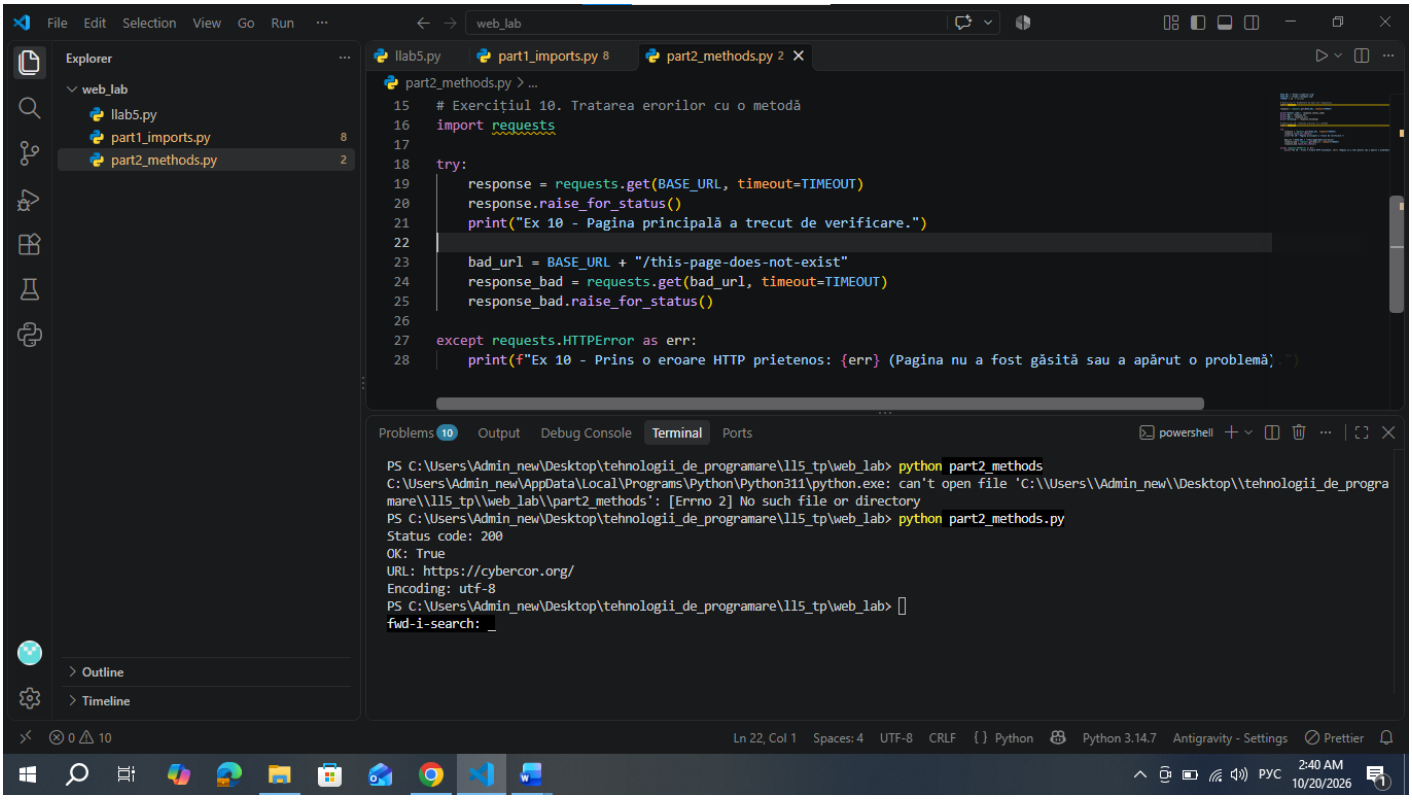


Figura 2.2 - Tratarea erorilor cu o metodă

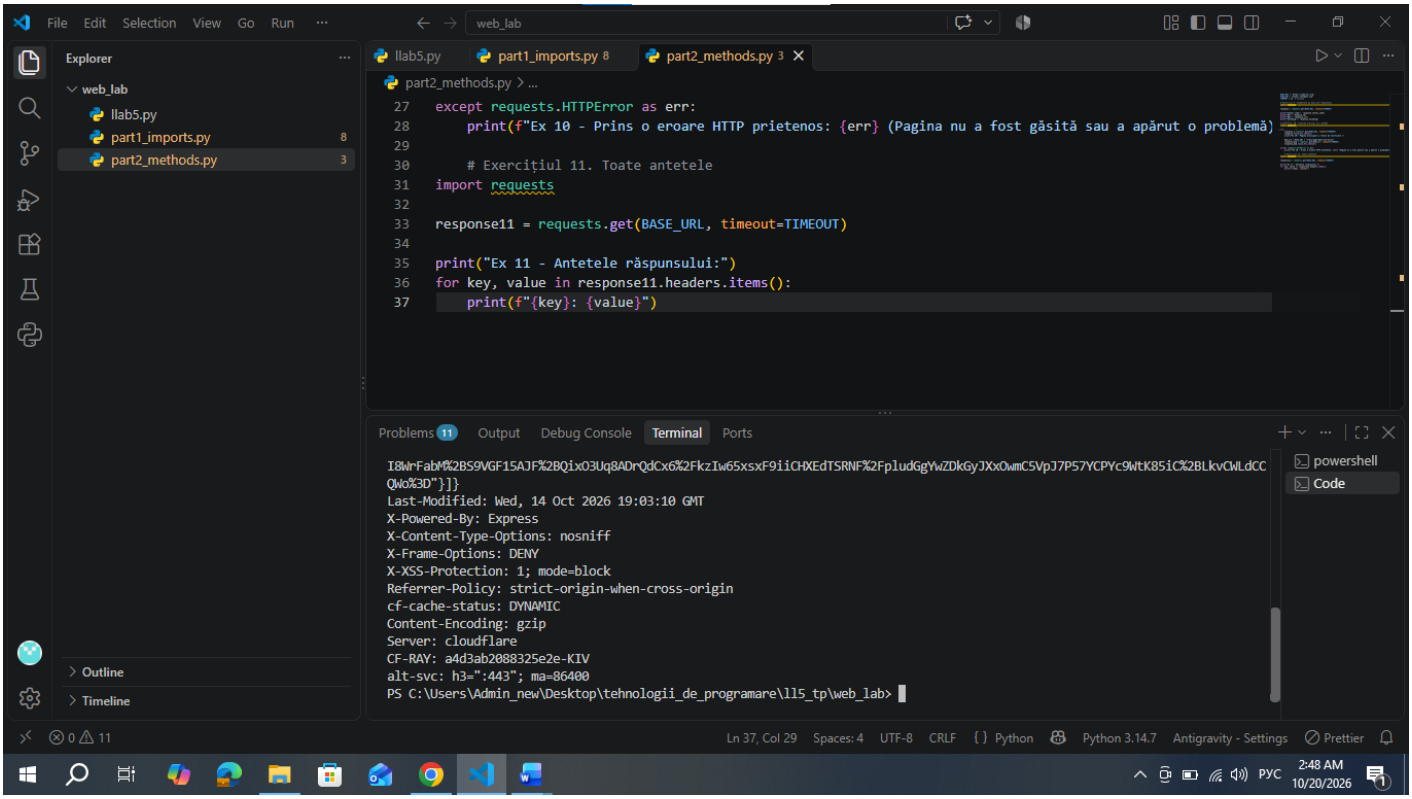


Figura 2.3 - Toate antetele

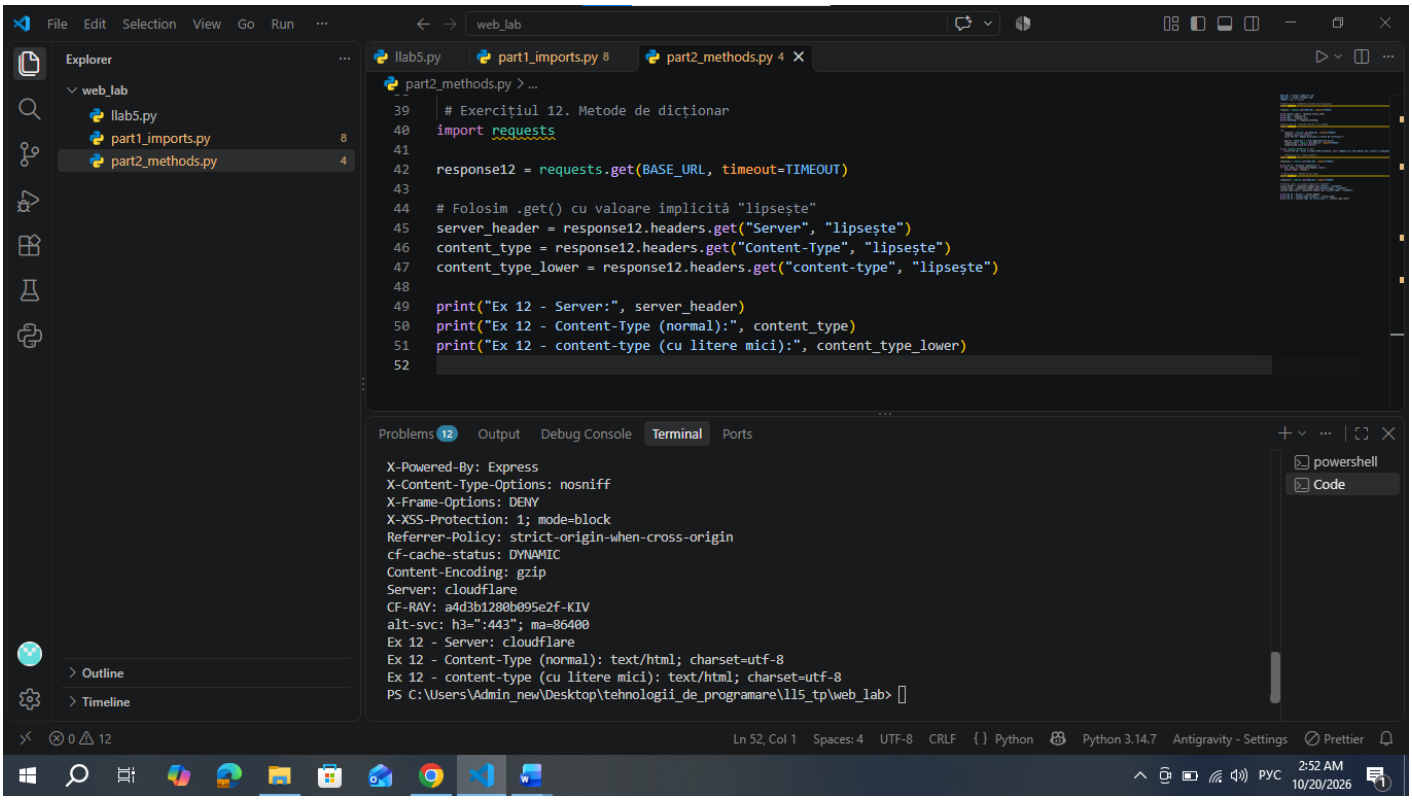


Figura 2.4 - Metode de dicționar

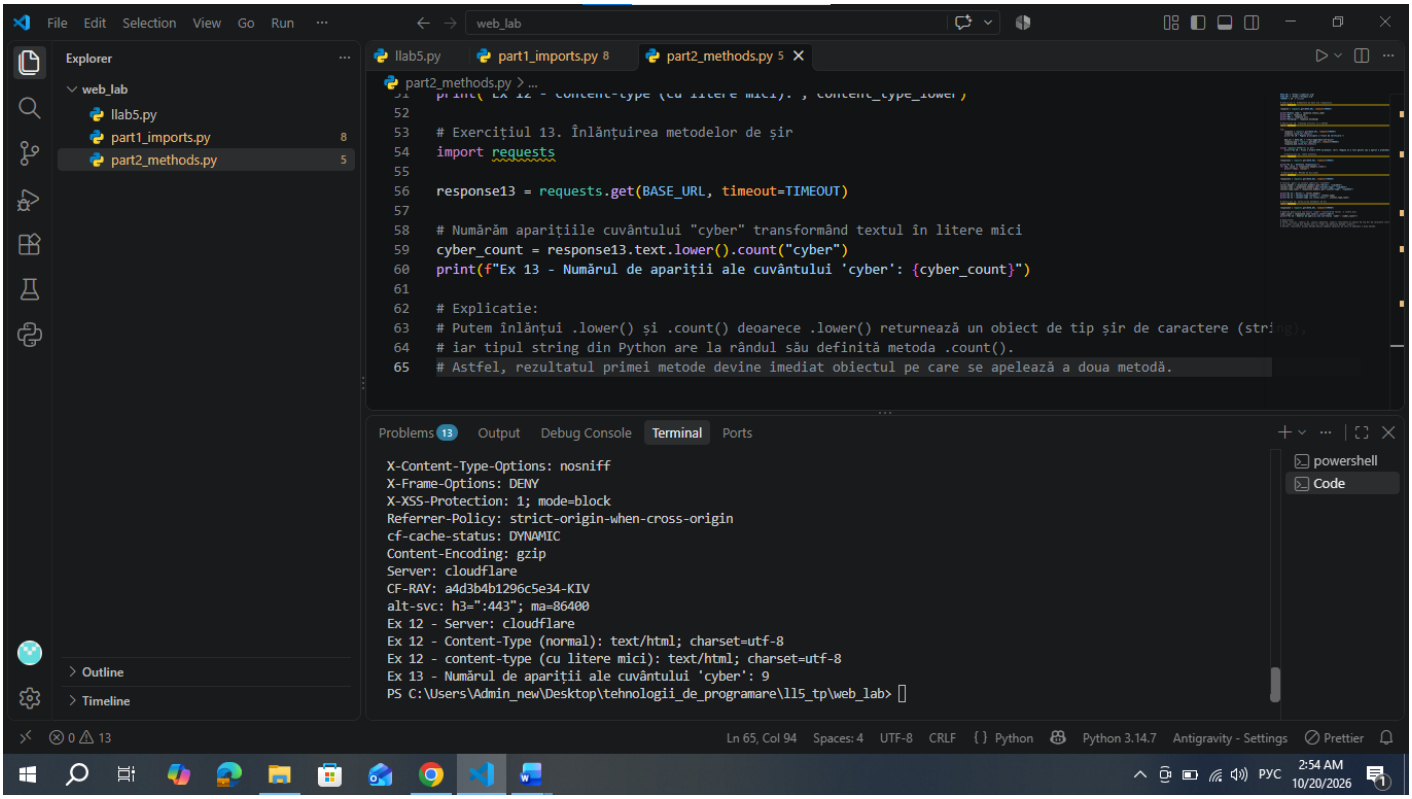


Figura 2.5 - Înlănțuirea metodelor de șir

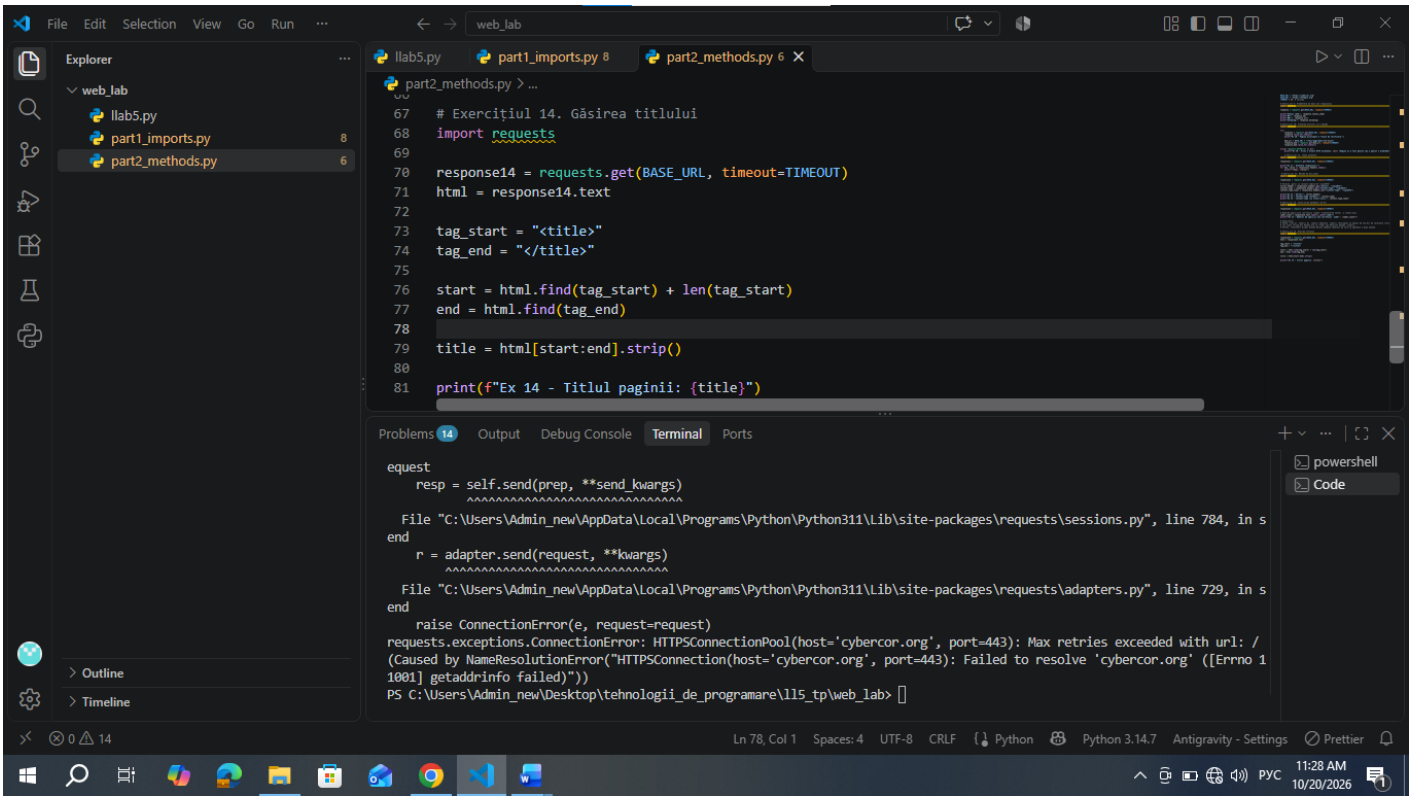


Figura 2.6 - Găsirea titlului

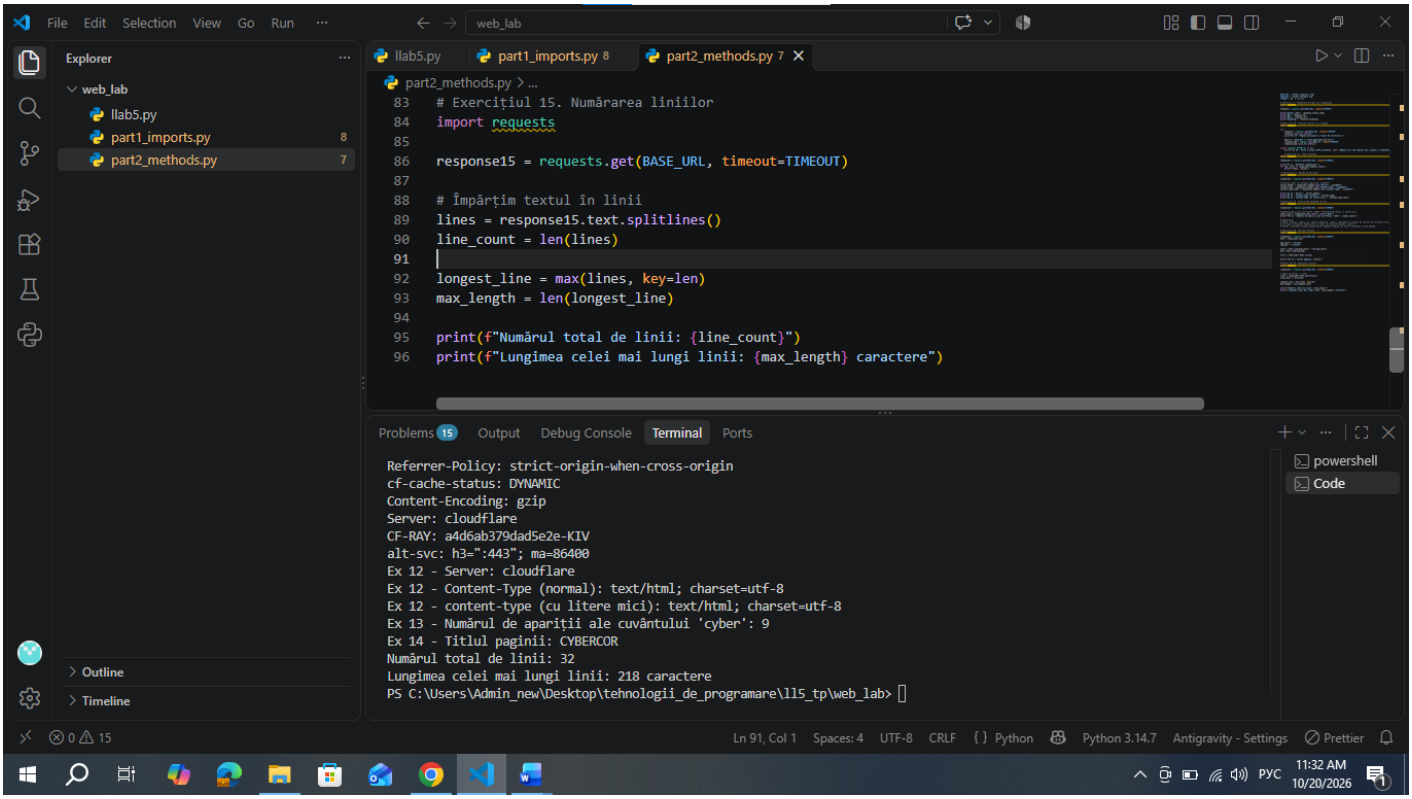


Figura 2.7 - Numărarea liniilor

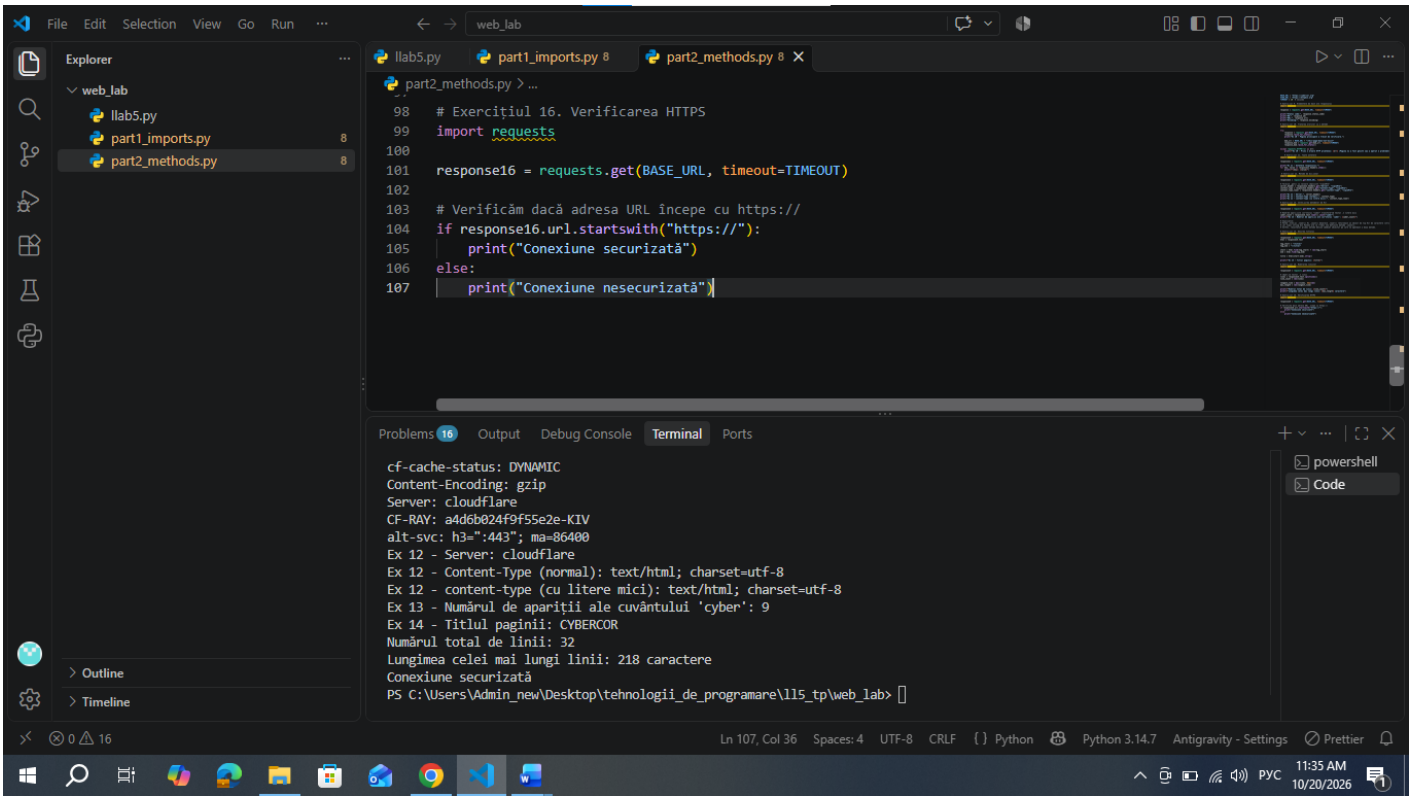


Figura 2.8 - Verificarea HTTPS

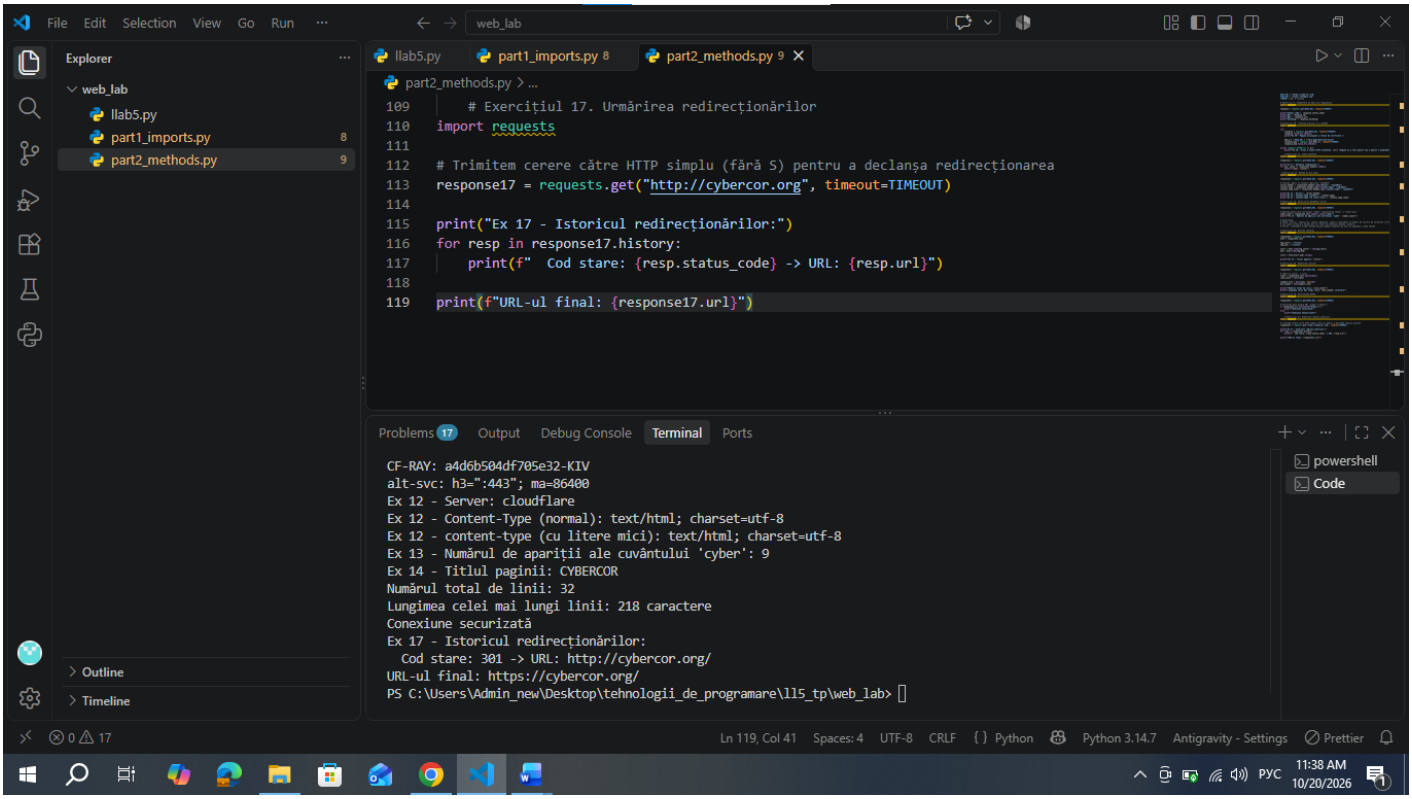


Figura 2.9 - Urmărirea redirecționărilor

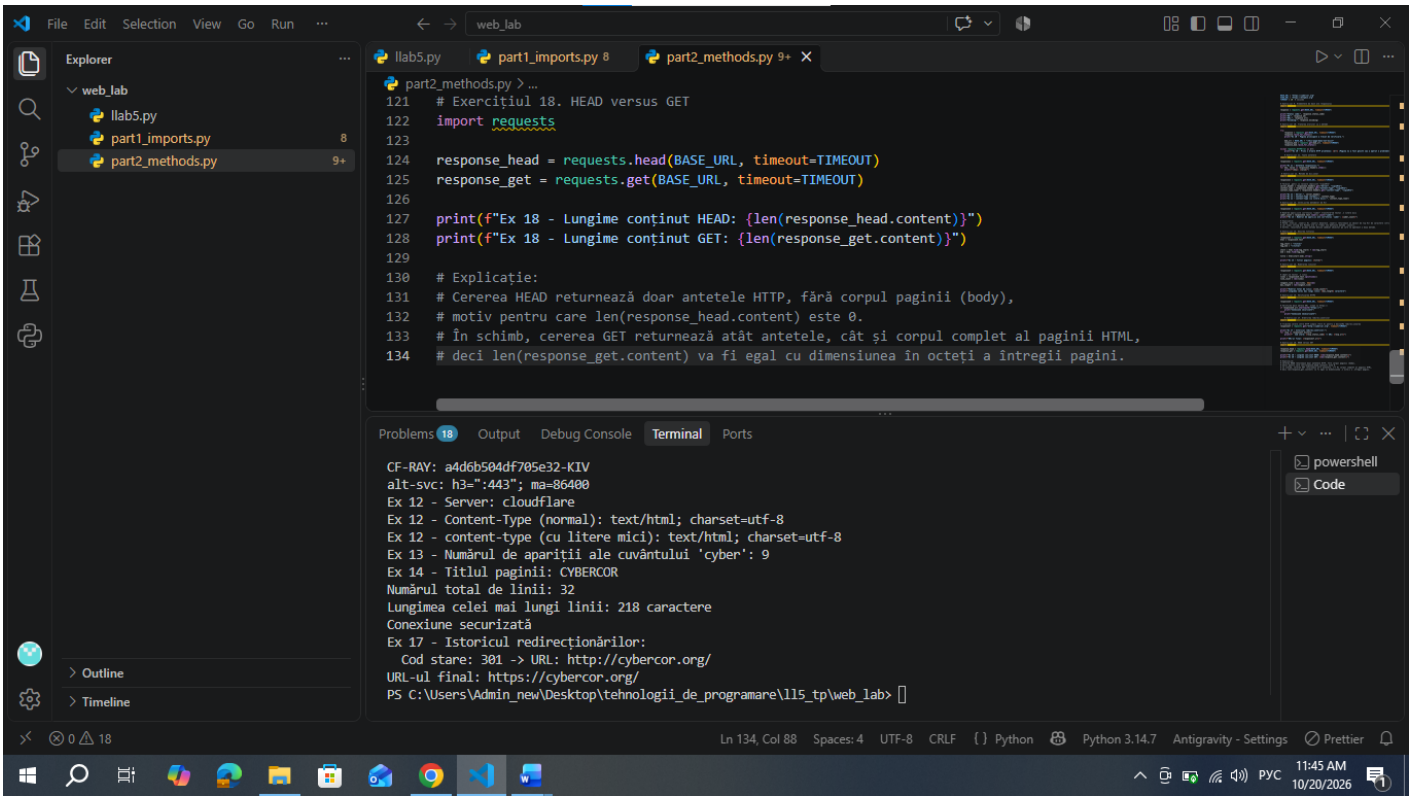


Figura 2.10 - HEAD versus GET

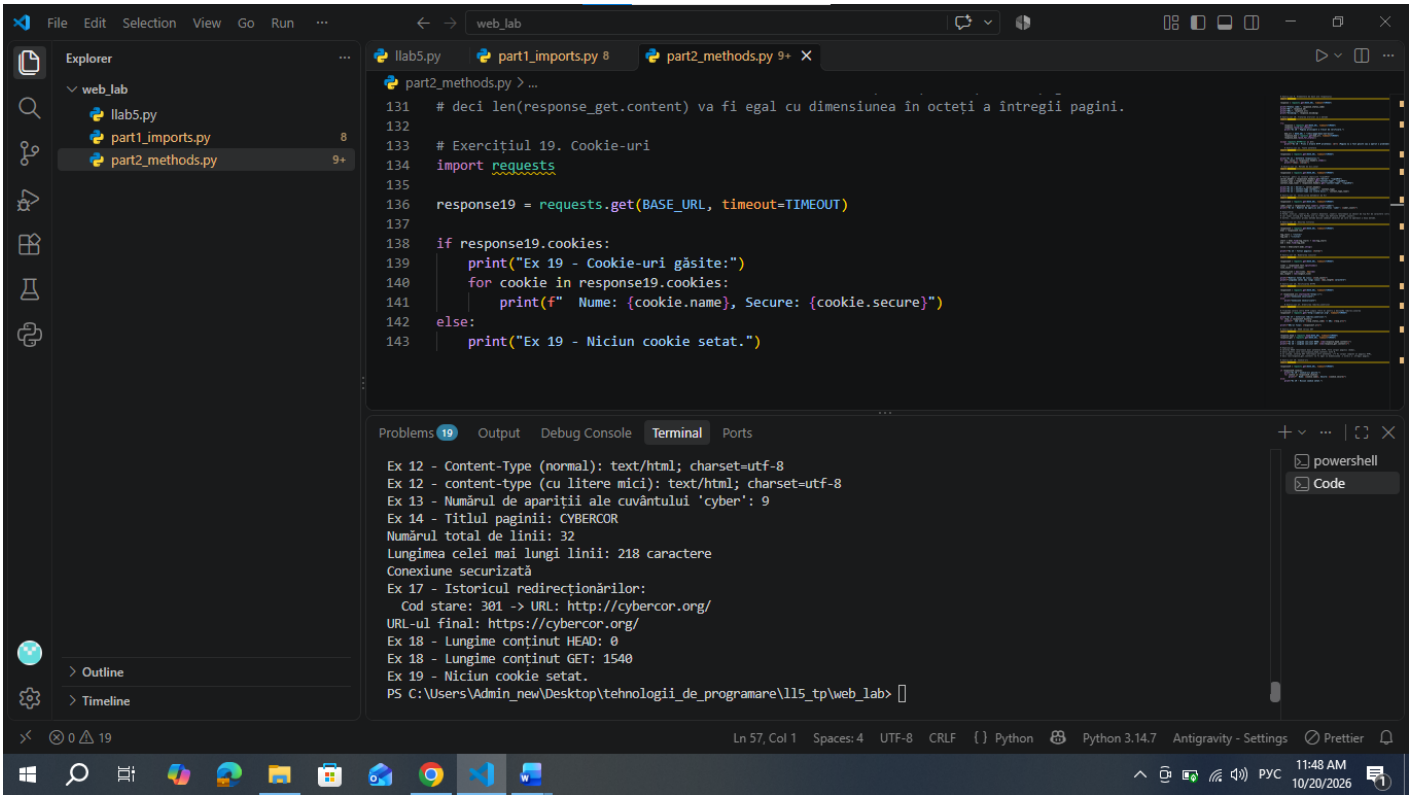


Figura 2.11 - Cookie-uri

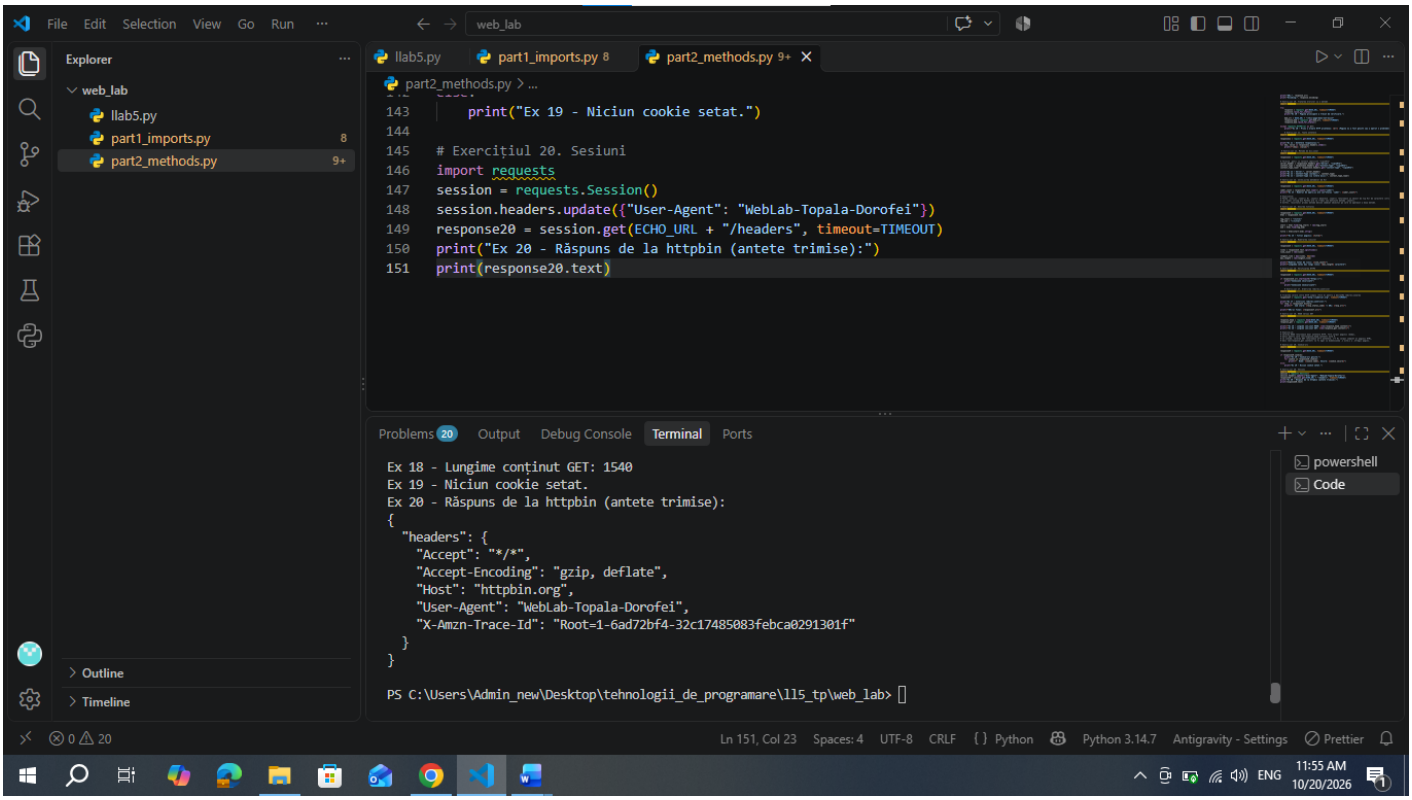


Figura 2.12 – Sesiuni

3. Partea 3: Scrierea propriilor

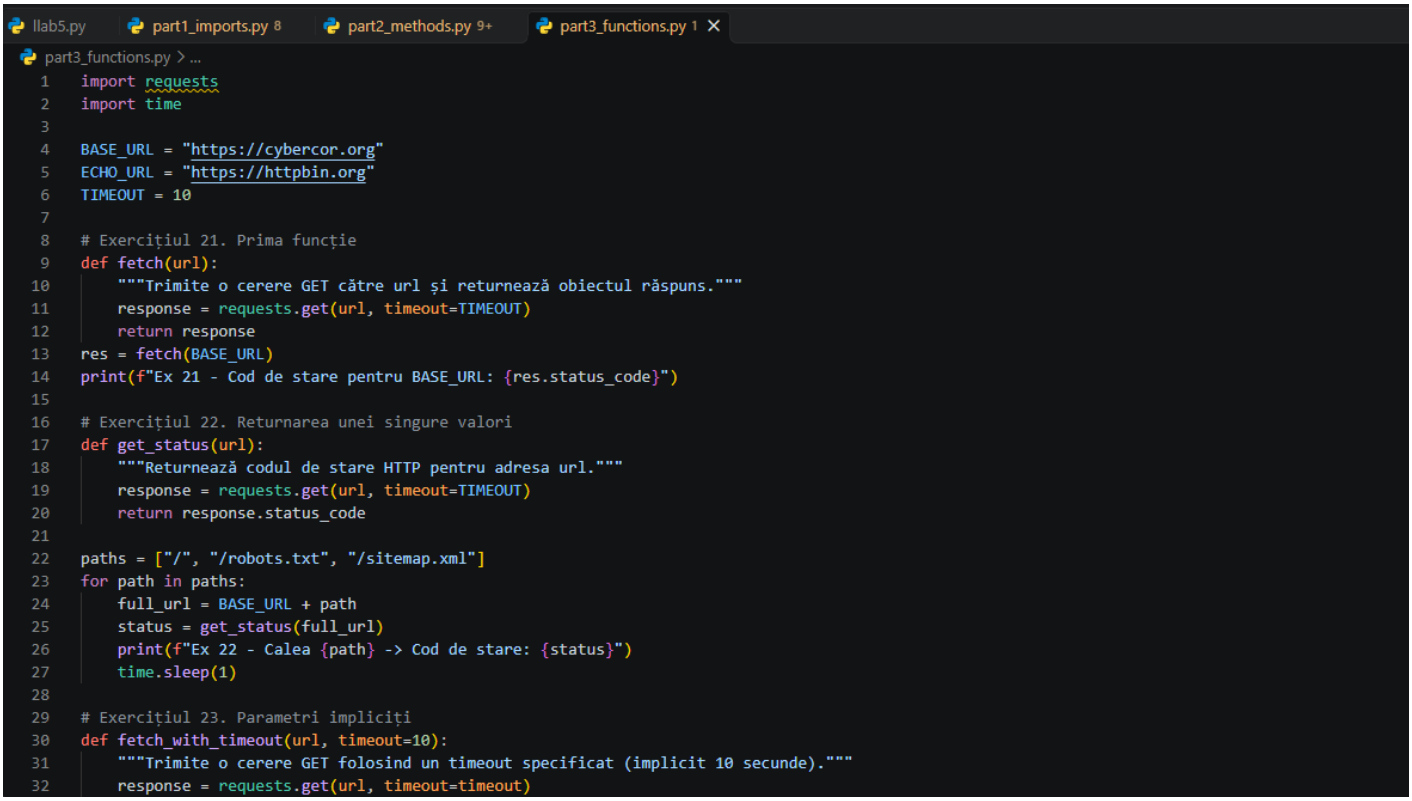


Figura 3.1 - Prima funcție, Returnarea unei singure valori, Parametri impliciți

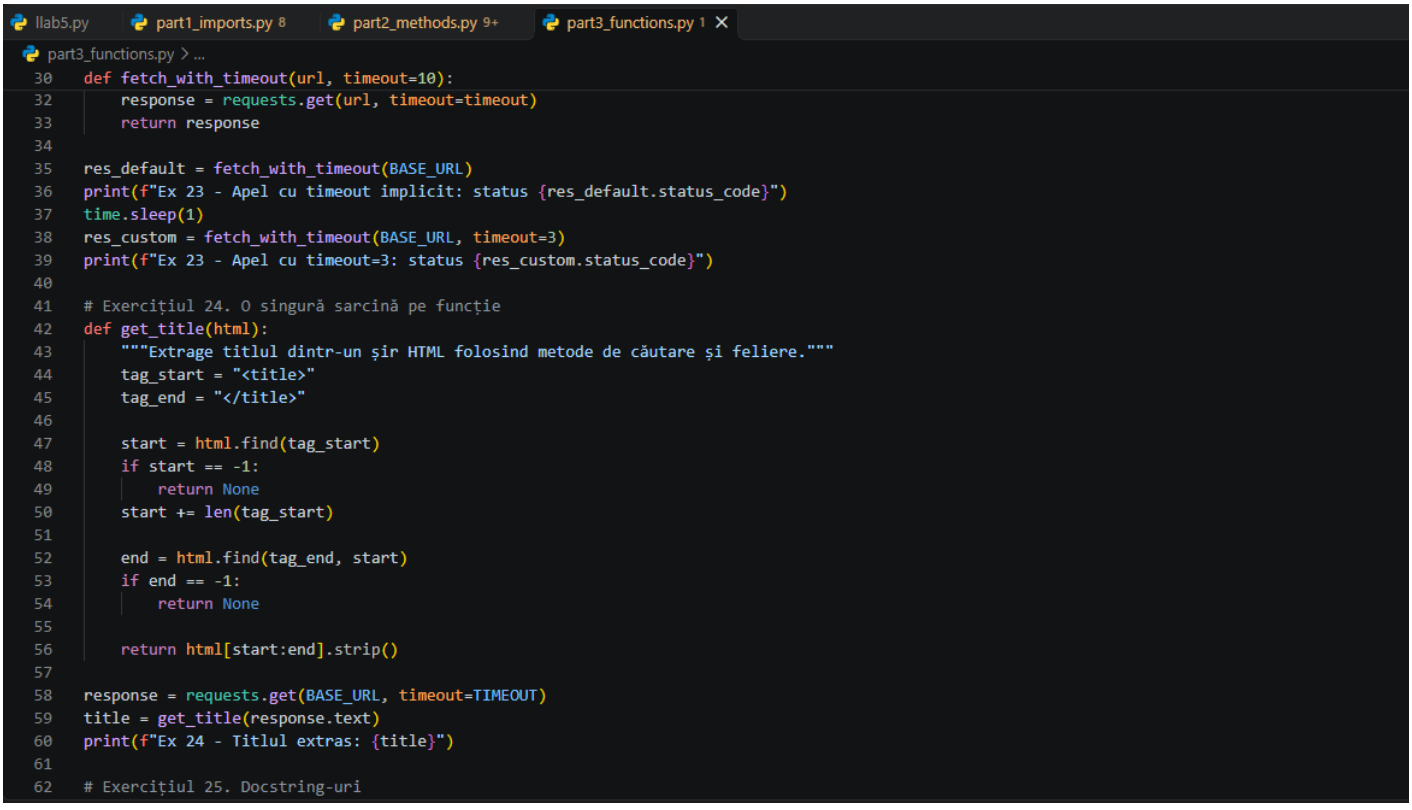


Figura 3.2 - O singură sarcină pe funcție

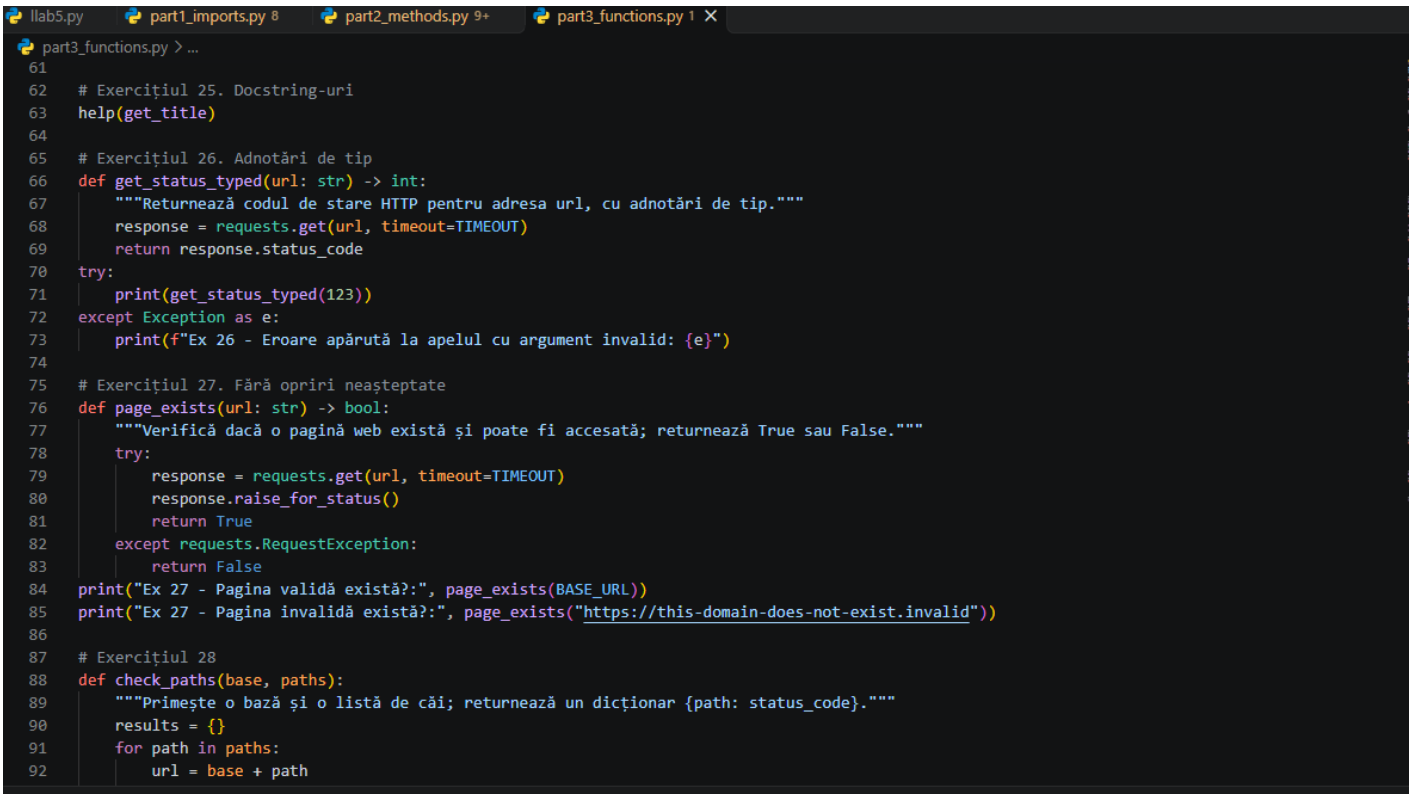


Figura 3.3 - Docstring-uri, Adnotări de tip, Fără opriri neașteptate, Returnarea unui dicționar

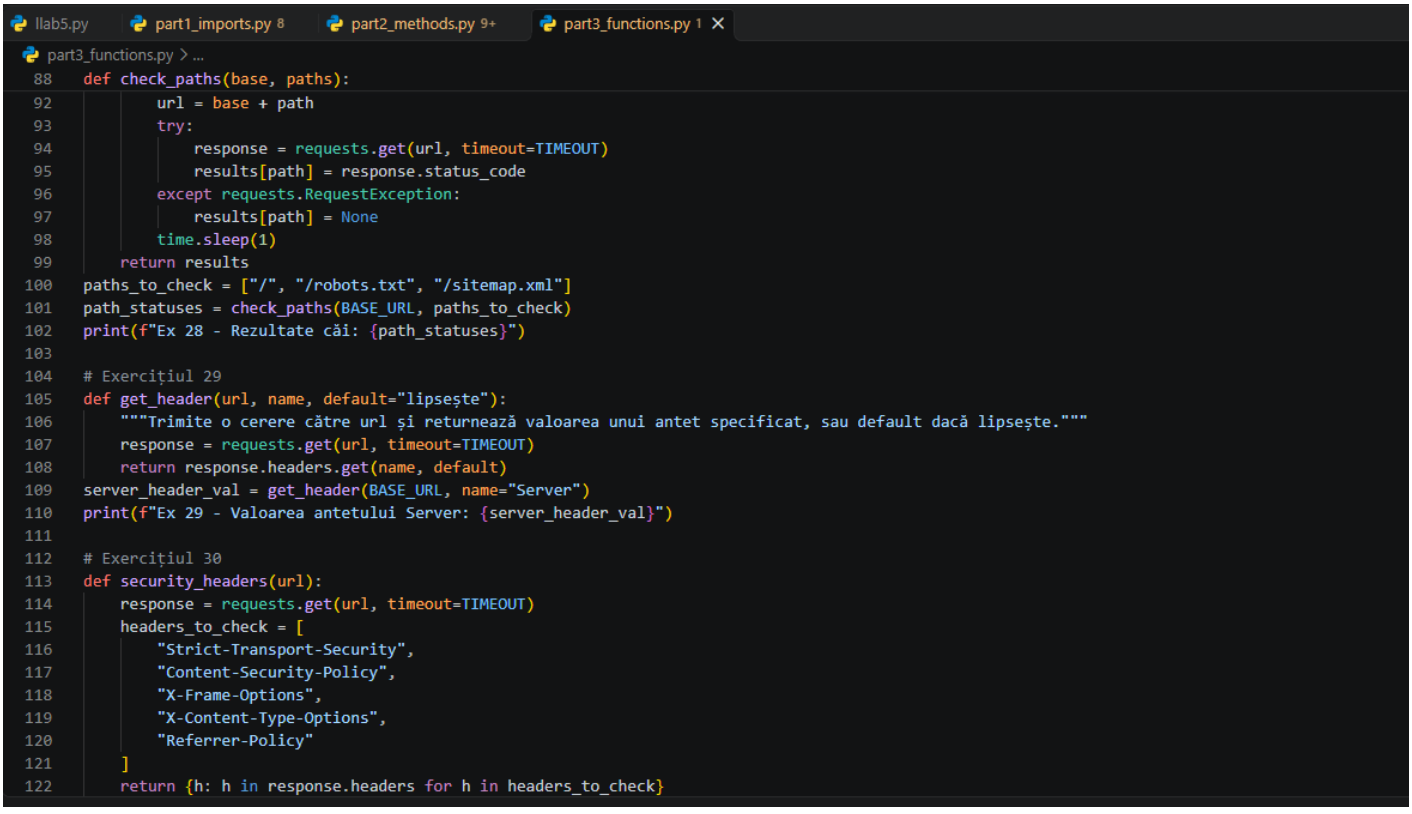


Figura 3.4 - Argumente cu nume, Antete de securitate

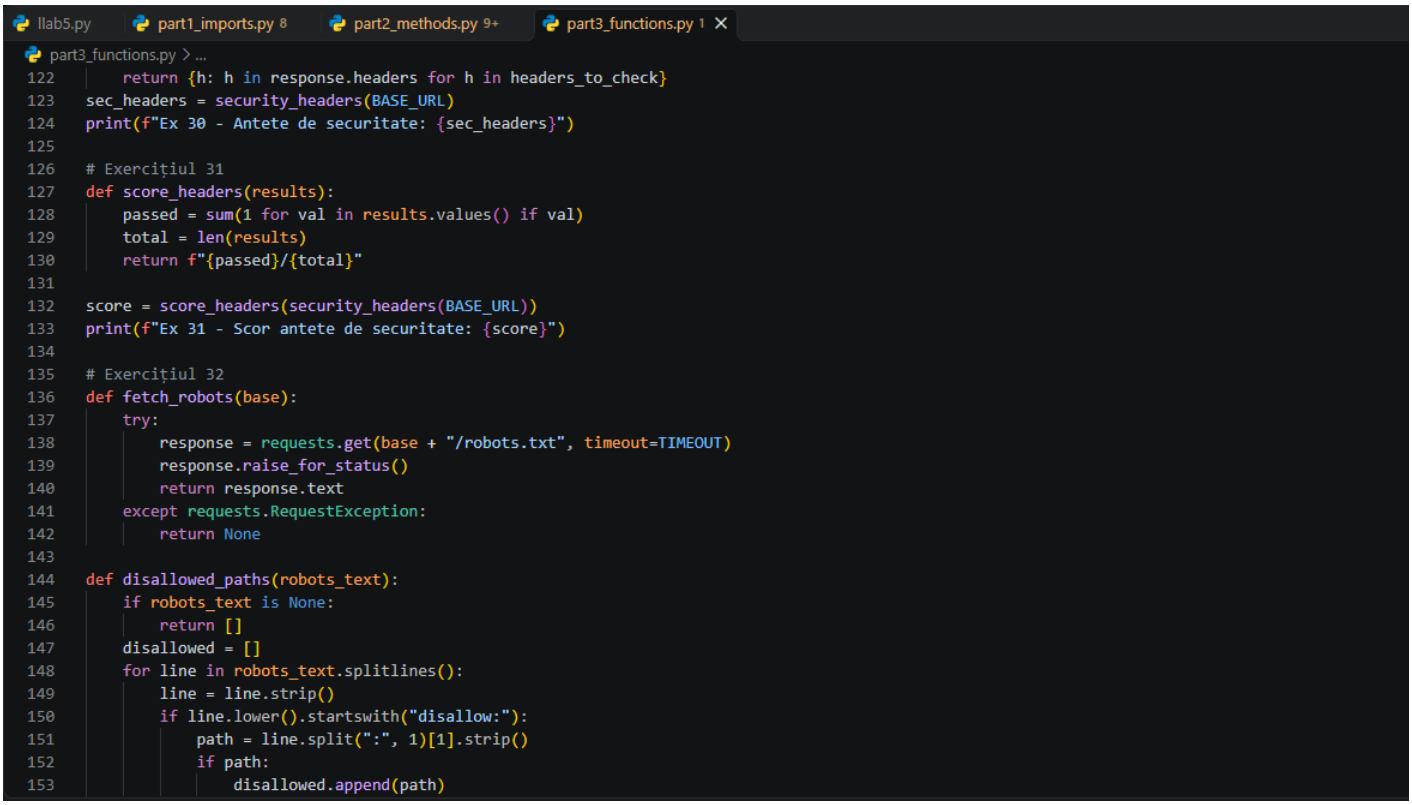


Figura 3.5 - Funcții care apelează funcții, Două funcții, o sarcină

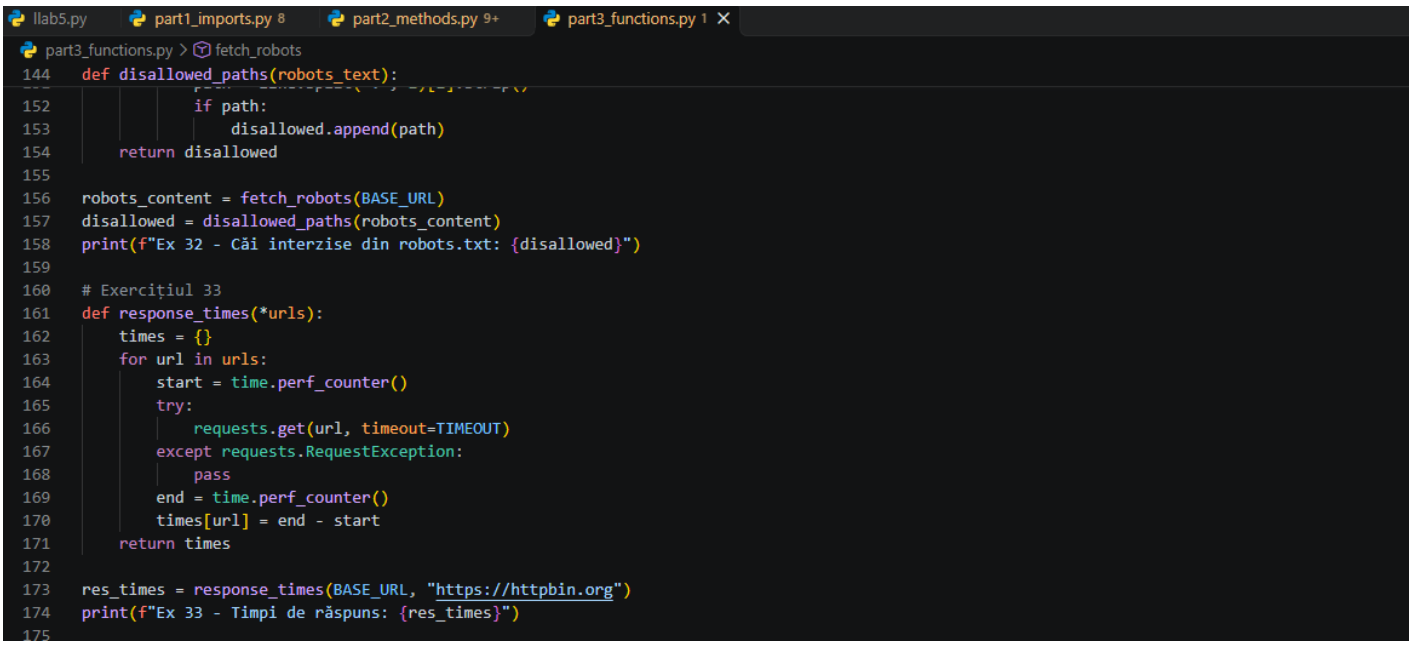


Figura 3.6 - Oricâte argumente

```python
# Exercițiul 34
def log(message, **details):
    """Afișează un mesaj însoțit de detalii suplimentare colectate ca argumente cu nume."""
    parts = [message]
    for key, value in details.items():
        parts.append(f"{key}={value}")
    print(" | ".join(parts))

# Testarea funcției
log("verificat", url=BASE_URL, status=200)
```

Figura 3.7 - Argumente cu nume colectate

```
PS C:\Users\Admin_new\Desktop\tehnologii_de_programare\ll5_tp\web_lab> python -u "c:\Users\Admin_new\Desktop\tehnologii_de_programare\ll5_tp\web_lab\part3_functions.
py"
Ex 21 - Cod de stare pentru BASE_URL: 200
Ex 22 - Calea / -> Cod de stare: 200
Ex 22 - Calea /robots.txt -> Cod de stare: 200
Ex 22 - Calea /sitemap.xml -> Cod de stare: 200
Ex 23 - Apel cu timeout implicit: status 200
Ex 23 - Apel cu timeout=3: status 200
Ex 24 - Titlul extras: CYBERCOR
Help on function get_title in module __main__:

get_title(html)
    Extrage titlul dintr-un \u0219ir HTML folosind metode de c\u0103utare \u0219i feliere.

Ex 26 - Eroare apărută la apelul cu argument invalid: Invalid URL '123': No scheme supplied. Perhaps you meant https://123?
Ex 27 - Pagina validă există?: True
Ex 27 - Pagina invalidă există?: False
Ex 28 - Rezultate căi: {'/': 200, '/robots.txt': 200, '/sitemap.xml': 200}
Ex 29 - Valoarea antetului Server: cloudflare
Ex 30 - Antete de securitate: {'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': True, 'X-Content-Type-Options': True, 'Referr
er-Policy': True}
Ex 31 - Scor antete de securitate: 3/5
Ex 32 - Căi interzise din robots.txt: []
Ex 33 - Timpi de răspuns: {'https://cybercor.org': 0.6995292000938207, 'https://httpbin.org': 1.4661904000677168}
verificat | url=https://cybercor.org | status=200
PS C:\Users\Admin_new\Desktop\tehnologii_de_programare\ll5_tp\web_lab>
```

Figura 3.8 – ce ne arată la terminal

4. Partea 4: Alte module din biblioteca standard



Figura 4.1 - Descompunerea unui URL, Compunerea URL-urilor

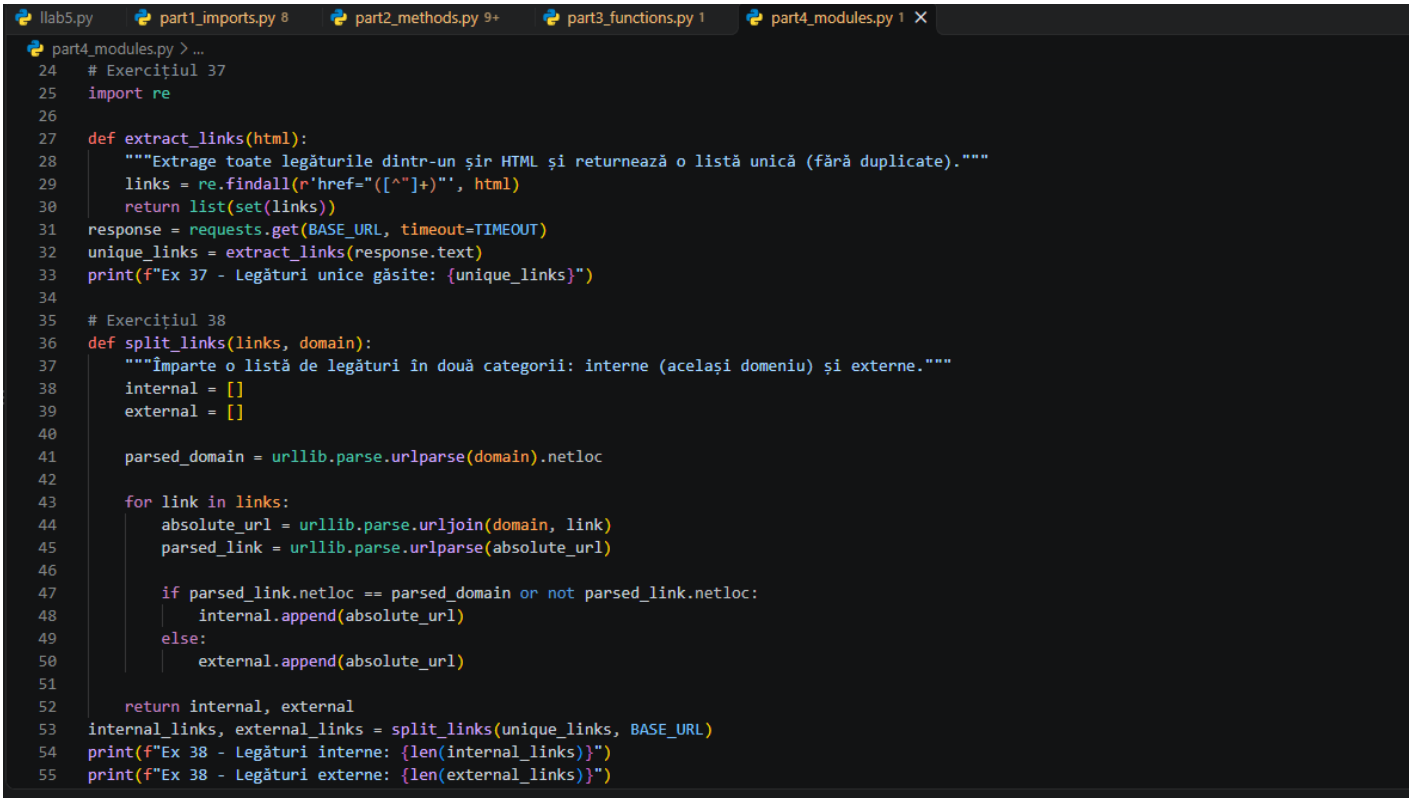


Figura 4.2 - Extragerea legăturilor, Interne sau externe?

```python
# Exercițiul 39
from html.parser import HTMLParser

class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        if tag.lower() == "img":
            src = dict(attrs).get("src")
            if src:
                self.images.append(src)

test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print("Ex 39 - Test imagini:", finder.images)
assert finder.images == [
    "/logo.png",
    "poza.jpg",
    "https://cdn.example.com/banner.webp",
```

Figura 4.3 - Propriile metode

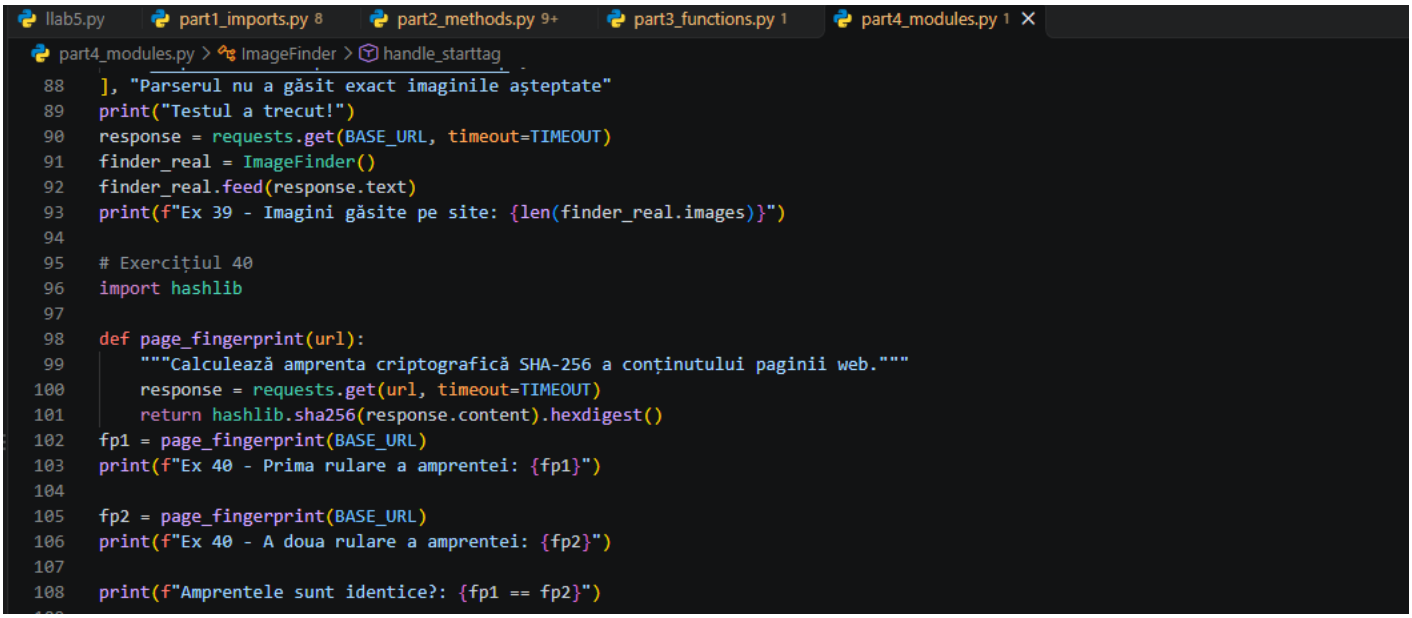


Figura 4.4 - Amprenta paginii

```python

# Exercițiul 41
import json

response = requests.get(BASE_URL, timeout=TIMEOUT)
with open("headers.json", "w", encoding="utf-8") as f:
    json.dump(dict(response.headers), f, indent=2)
with open("headers.json", "r", encoding="utf-8") as f:
    loaded_headers = json.load(f)
server_val = loaded_headers.get("Server", "lipsește")
print(f"Ex 41 - Antetul Server încărcat din JSON: {server_val}")

# Exercițiul 42. Interogare DNS
import socket

def resolve(hostname):
    """Returnează adresa IP pentru un hostname dat folosind modulul socket."""
    return socket.gethostbyname(hostname)
ip_address = resolve("cybercor.org")
print(f"Ex 42 - Adresa IP pentru cybercor.org: {ip_address}")

```

Figura 4.5 - Salvarea în JSON, Interogare DNS

```python
# Exercițiul 43. Expirarea certificatului
import ssl
from datetime import datetime, timezone

def cert_days_left(hostname):
    """Calculează câte zile mai sunt până la expirarea certificatului SSL al unui hostname."""
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443), timeout=TIMEOUT) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as ssock:
            cert = ssock.getpeercert()
            not_after_str = cert["notAfter"]
            expiry_seconds = ssl.cert_time_to_seconds(not_after_str)
            expiry_date = datetime.fromtimestamp(expiry_seconds, tz=timezone.utc)
            current_date = datetime.now(timezone.utc)
            delta = expiry_date - current_date
            return delta.days
days_left = cert_days_left("cybercor.org")
print(f"Ex 43 - Zile rămase până la expirarea certificatului: {days_left} zile")
```

Figura 4.6 - Expirarea certificatului

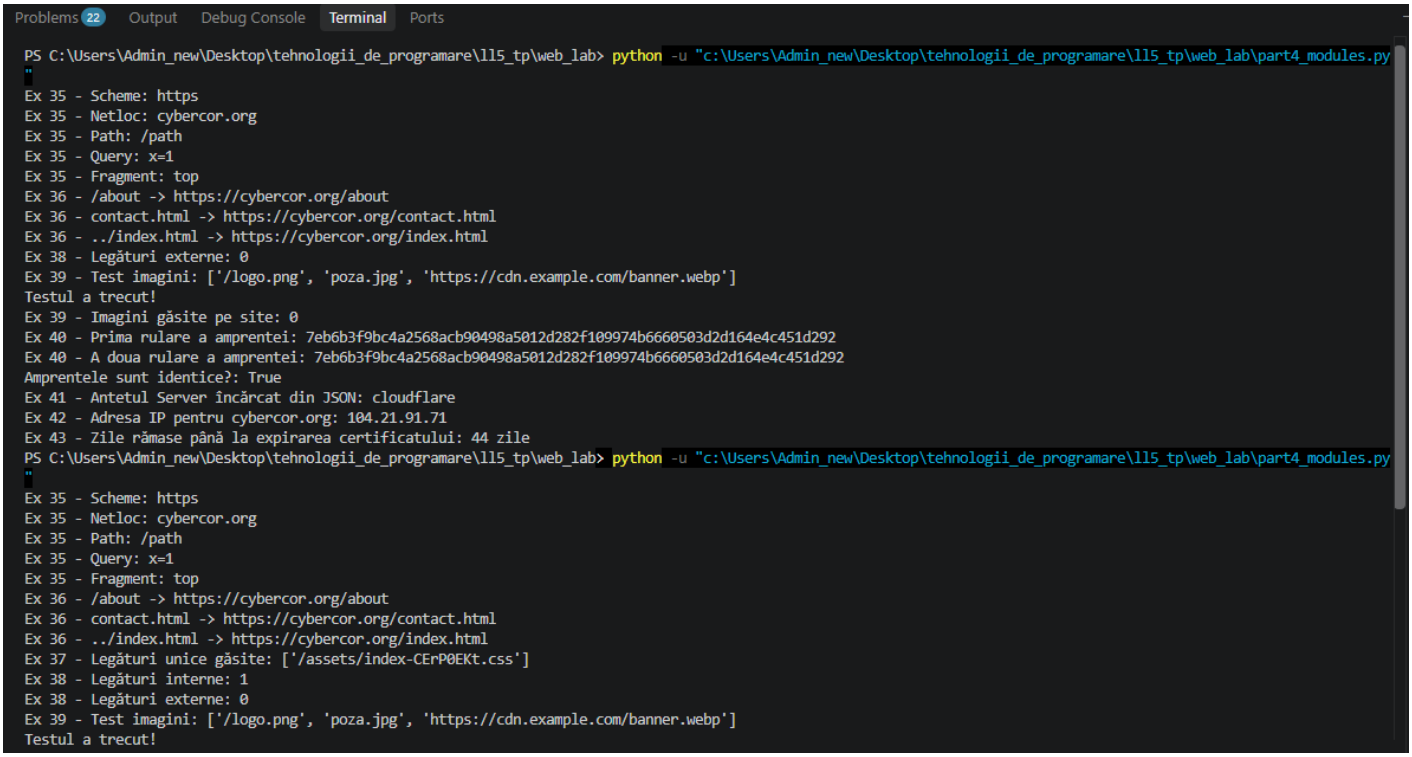


Figura 4.7.1 – Ce ne arată terminalul

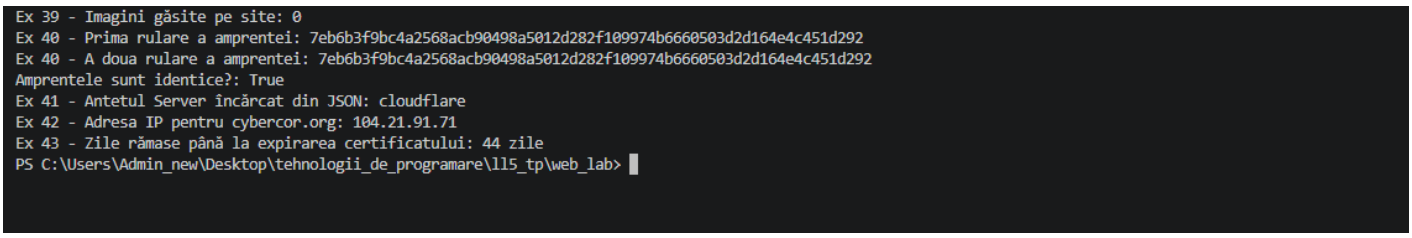


Figura 4.7.2 – Ce ne arată terminalul

5. Partea 5: Construirea propriului modul

part1_imports.py 8 | part2_methods.py 9+ | part3_functions.py 1 | part4_modules.py 1 | webtools.cpython-311.pyc | webtools.py 1 | main.py

webtools.py > fetch

```python
# Modulul webtools.py
import requests

BASE_URL = "https://cybercor.org"
TIMEOUT = 10

def fetch(url, timeout=TIMEOUT):
    """Returnează obiectul răspuns pentru un URL dat."""
    return requests.get(url, timeout=timeout)

def get_status(url):
    """Returnează codul de stare HTTP pentru adresa url."""
    response = requests.get(url, timeout=TIMEOUT)
    return response.status_code

def get_title(html):
    """Extrage titlul dintr-un șir HTML folosind metode de căutare și feliere."""
    tag_start = "<title>"
    tag_end = "</title>"
    start = html.find(tag_start)
    if start == -1:
        return None
    start += len(tag_start)
    end = html.find(tag_end, start)
    if end == -1:
        return None
    return html[start:end].strip()
```

Figura 5.1.1 - Web tools

webtools.py > get_title

```python
def security_headers(url):
    """Verifică prezența antetelor de securitate și returnează un dicționar {antet: True/False}."""
    response = requests.get(url, timeout=TIMEOUT)
    headers_to_check = [
        "Strict-Transport-Security",
        "Content-Security-Policy",
        "X-Frame-Options",
        "X-Content-Type-Options",
        "Referrer-Policy"
    ]
    return {h: h in response.headers for h in headers_to_check}

if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))

DEFAULT_HEADERS = {"User-Agent": "WebLab-Dorofei-Topala"}

def fetch(url, timeout=TIMEOUT):
    """Returnează obiectul răspuns pentru un URL dat, folosind antetele implicite."""
    return requests.get(url, headers=DEFAULT_HEADERS, timeout=timeout)

def _format_url(url):
    """Asigură că URL-ul începe cu o schemă validă (http:// sau https://)."""
    if not url.startswith(("http://", "https://")):
        return "https://" + url
    return url

def fetch(url, timeout=TIMEOUT):
    """Returnează obiectul răspuns pentru un URL dat, formatat și folosind antetele implicite."""
    formatted_url = _format_url(url)
    return requests.get(formatted_url, headers=DEFAULT_HEADERS, timeout=timeout)
```

Figura 5.1.2 - Web tools

part1_imports.py 8 | part2_methods.py 9+ | part3_functions.py 1 | part4_modules.py 1 | webtools.cpython-311.pyc | webtools.py 1 | main.py

main.py > ...

```python
# Programul principal main.py - Exercițiul 50 (Tabloul de bord web)
import webtools
def audit_dashboard(url):
    print("=" * 50)
    print(f" RAPORT DE AUDIT PENTRU: {url}")
    print("=" * 50)
    try:
        response = webtools.fetch(url)
        print(f"[+] Status HTTP: {response.status_code}")

        title = webtools.get_title(response.text)
        print(f"[+] Titlul paginii: {title if title else 'N/A'}")

        sec_headers = webtools.security_headers(url)
        print("[+] Antete de securitate detectate:")
        passed = 0
        for header, present in sec_headers.items():
            status_icon = "✓" if present else "X"
            print(f"    [{status_icon}] {header}: {present}")
            if present:
                passed += 1

        score = f"{passed}/{len(sec_headers)}"
        print(f"[+] Scor securitate antete: {score}")

    except Exception as e:
        print(f"[-] Eroare la efectuarea auditului: {e}")

    print("=" * 50)

if __name__ == "__main__":
    audit_dashboard(webtools.BASE_URL)
```

Figura 5.2 - Main

```
PS C:\Users\Admin_new\Desktop\tehnologii_de_programare\l15_tp\web_lab> python main.py
==================================================
 RAPORT DE AUDIT PENTRU: https://cybercor.org
==================================================
[+] Status HTTP: 200
[+] Titlul paginii: CYBERCOR
[+] Antete de securitate detectate:
    [X] Strict-Transport-Security: False
    [X] Content-Security-Policy: False
    [✓] X-Frame-Options: True
    [✓] X-Content-Type-Options: True
    [✓] Referrer-Policy: True
[+] Scor securitate antete: 3/5
==================================================
PS C:\Users\Admin_new\Desktop\tehnologii_de_programare\l15_tp\web_lab>
```

Figura 5.3 – Ce ne arată terminalul

CONCLUZII

În urma efectuării acestei lucrări de laborator, am aprofundat cunoștințele de programare web în Python. Am învățat cum să interacționăm cu paginile web folosind biblioteca requests, cum să extragem date utile cu ajutorul expresiilor regulate și parserelor HTML, și cum să verificăm elemente importante de securitate, cum ar fi antetele HTTP și certificatele SSL. De asemenea, am învățat să organizăm codul în module proprii (webtools.py) și să creăm un tablou de bord (dashboard) pentru auditul automat al unui site web.